Data Structures and Algorithms
Lecture – 7 Trees -Part II

M.G.Noel A.S Fernando (PhD)

Professor

UCSC

Efficient Binary Trees

Binary search trees

Efficient Binary Trees

- The efficient binary trees such as binary search trees, AVL trees, threaded binary trees, red-black trees, and splay trees.
- .Under the data structures and algorithms for Level II, Binary search trees and AVL trees will be discussed.

Binary Search trees

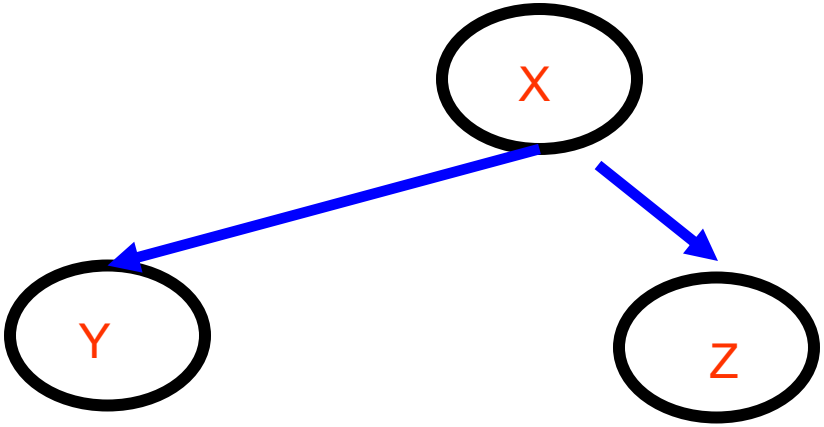
- A binary search tree, also known as an ordered binary tree, is a variant of binary trees in which the nodes are arranged in an order.
- In a binary search tree, all the nodes in the left sub-tree have a value less than that of the root node. Similarly, all the nodes in the right sub-tree have a value either equal to or greater than the root node.
- The same rule is applicable to every sub-tree in the tree.
- (Note that a binary search tree may or may not contain duplicate values, depending on its implementation.)

Binary Search Trees

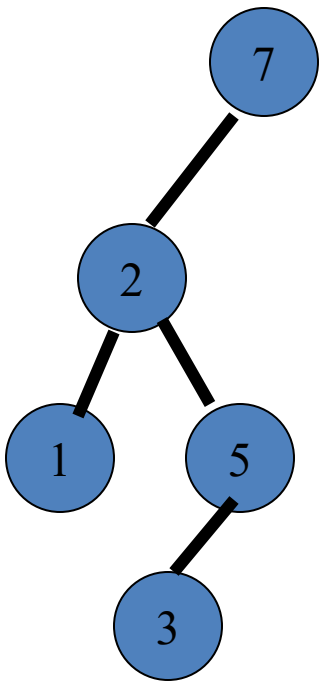
- In a binary search tree, each non-leaf node v stores an element e such that
  - the elements stored in the left sub tree of v are less than e.
  - the elements stored in the right sub tree of v are greater than e.

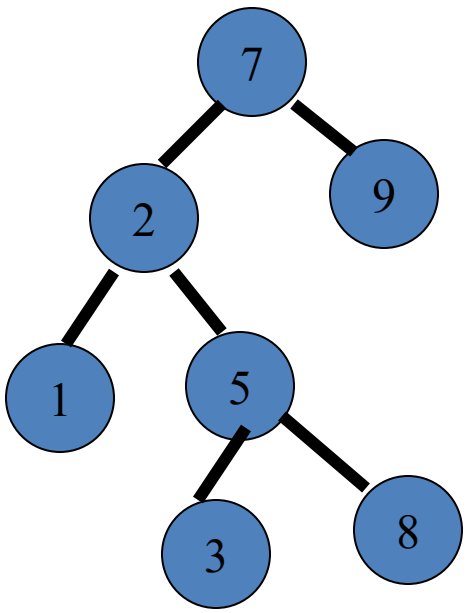
Binary Search Trees

- Key property
  - Value at node
    - Smaller values in left subtree
    - Larger values in right subtree
  - Example
    - X > Y
    - X < Z

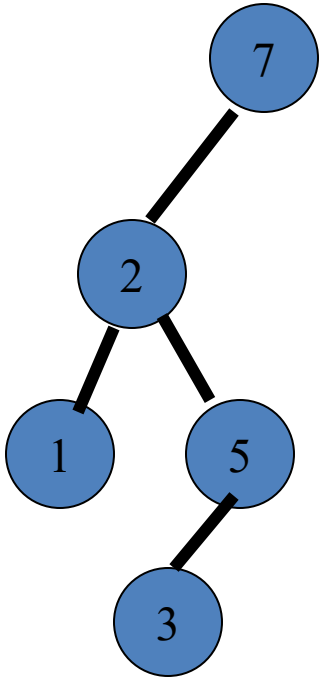


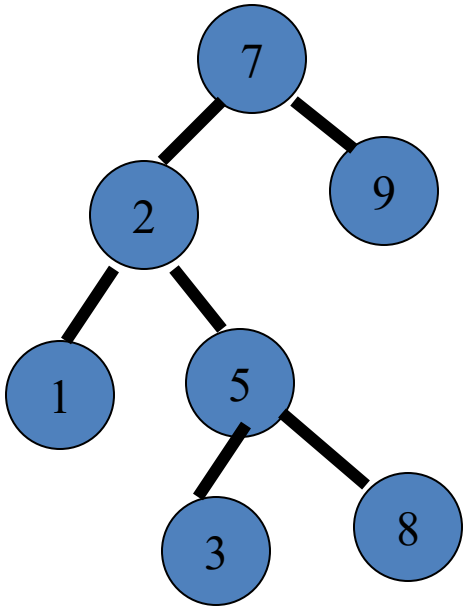
Examples :



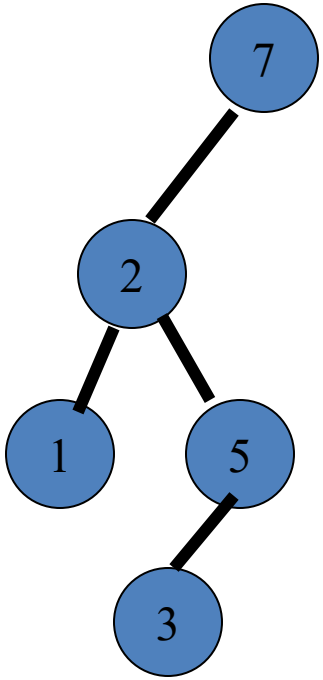


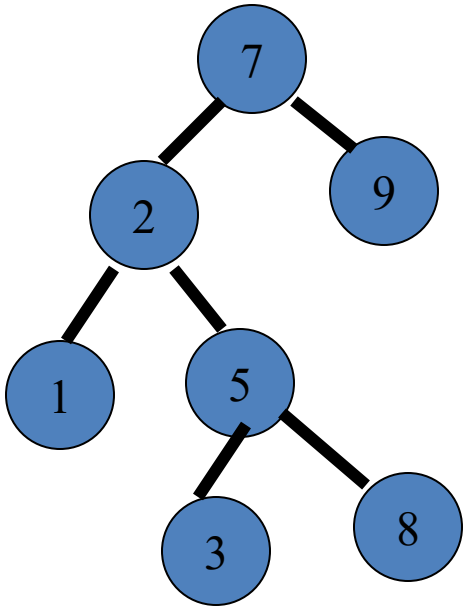
Examples :





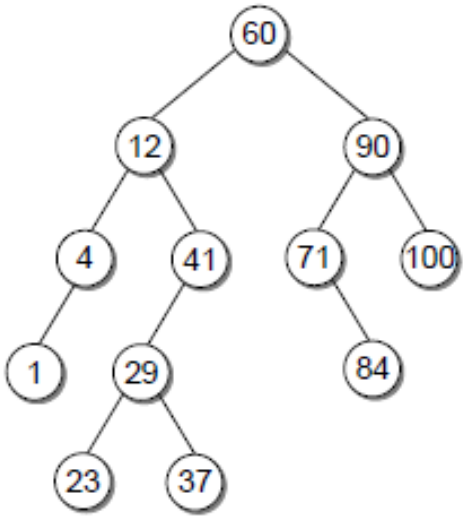
Examples :





Two binary trees(only the left tree is a search tree)

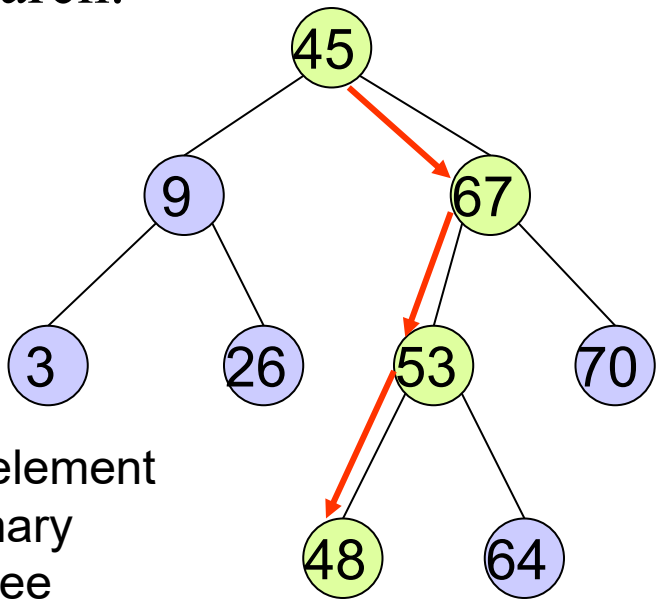
Binary Search Tree



Given the relationship between the nodes, an in-order traversal will visit the nodes in increasing search key order. For the example binary search tree, the order would be 1 4 12 23 29 37 41 60 71 84 90 100.

Binary Search Trees

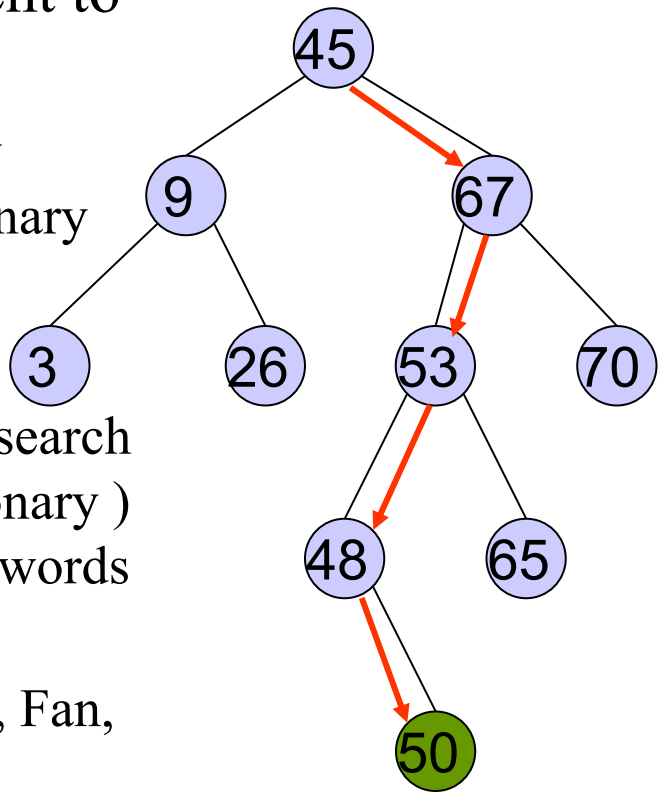
- Advantage of using binary search trees —support fast search.



Find an element in the binary search tree

Binary Search Trees

- Adding a new element to a binary search tree.
  - Example: add a new element 50 to the binary search tree.
  - Ex. Create a binary search tree (words of dictionary) using the given key words
  - Book, Sugar, River, Computer, Zoo, Jug, Fan, Pencil



Algorithm to search for a given value in a binary search tree

```
searchElement (TREE, VAL)

Step 1: IF TREE -> DATA = VAL OR TREE = NULL
          Return TREE
        ELSE
         IF VAL < TREE -> DATA
          Return searchElement(TREE -> LEFT, VAL)
         ELSE
          Return searchElement(TREE -> RIGHT, VAL)
         [END OF IF]
        [END OF IF]
Step 2: END
```

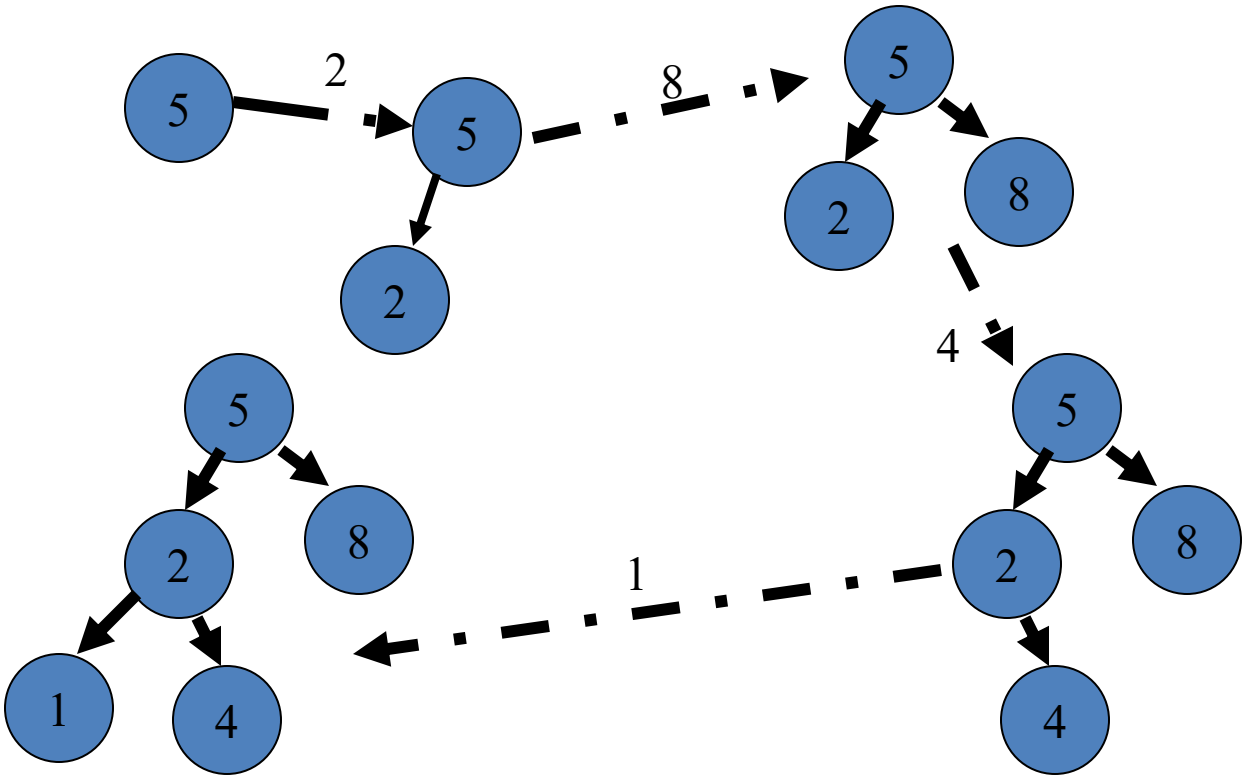
Applications : One way that a binary search tree can be used to sort data as follows.

- Eg: we have list of integers, such as 5,2,8,4, and 1 that we wish to sort in to ascending order.
- This algorithm consists of two stages.
- - Insertion
- -Traversal

Inserting- inserting integers into a binary search tree.

- **Step 1 :** If the current pointer is null. Create a new node, store the data , and return the address of the new node.
- Step 2 : otherwise compare the integer to the data stored at the current, if the new node is less than the integer at the current node, insert the new integer into the left child of the current node(by recursively applying the same algorithm), otherwise, insert it into the right child of the current node.

Eg : 5,2,8,4 and 1



Algorithm to insert a given value in a binary search tree

```
Insert (TREE, VAL)

Step 1: IF TREE = NULL
            Allocate memory for TREE
            SET TREE -> DATA = VAL
            SET TREE -> LEFT = TREE -> RIGHT = NULL
        ELSE
            IF VAL < TREE -> DATA
                  Insert(TREE -> LEFT, VAL)
            ELSE
                  Insert(TREE -> RIGHT, VAL)
            [END OF IF]
        [END OF IF]
Step 2: END
```

Algorithm to insert a given value in a binary search tree

Note :

- The insert function requires time proportional to the height of the tree in the worst case.
- It takes `O(log n)` time to execute in the average case and `O(n)` time in the worst case.

Binary Search Trees

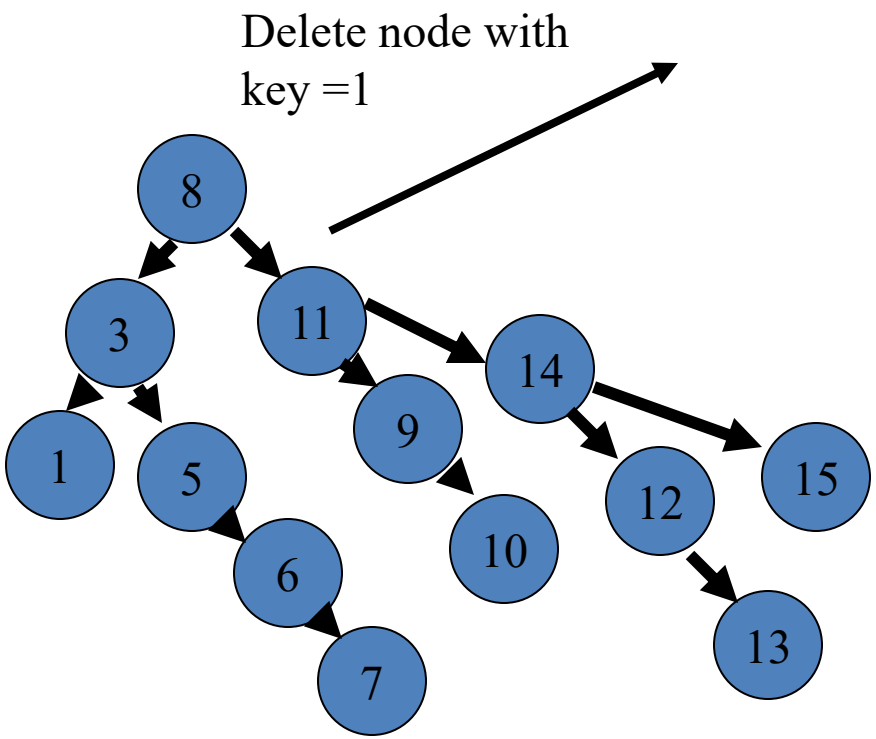
- Adding a new element to a binary search tree.

```
Input: binary search tree T, node v, element e
Output:
add(T, v, e){
        if(T.isLeaf(v)){
                if(v.element()>=e)
                        add element e as v's left child
                else
                        add element e as v's right child
        } else {
        if(v.element()>=e)
                        add(T, T.leftChild(v), e)
                else
                        add(T, T.rightChild(v), e)
    }
}
```

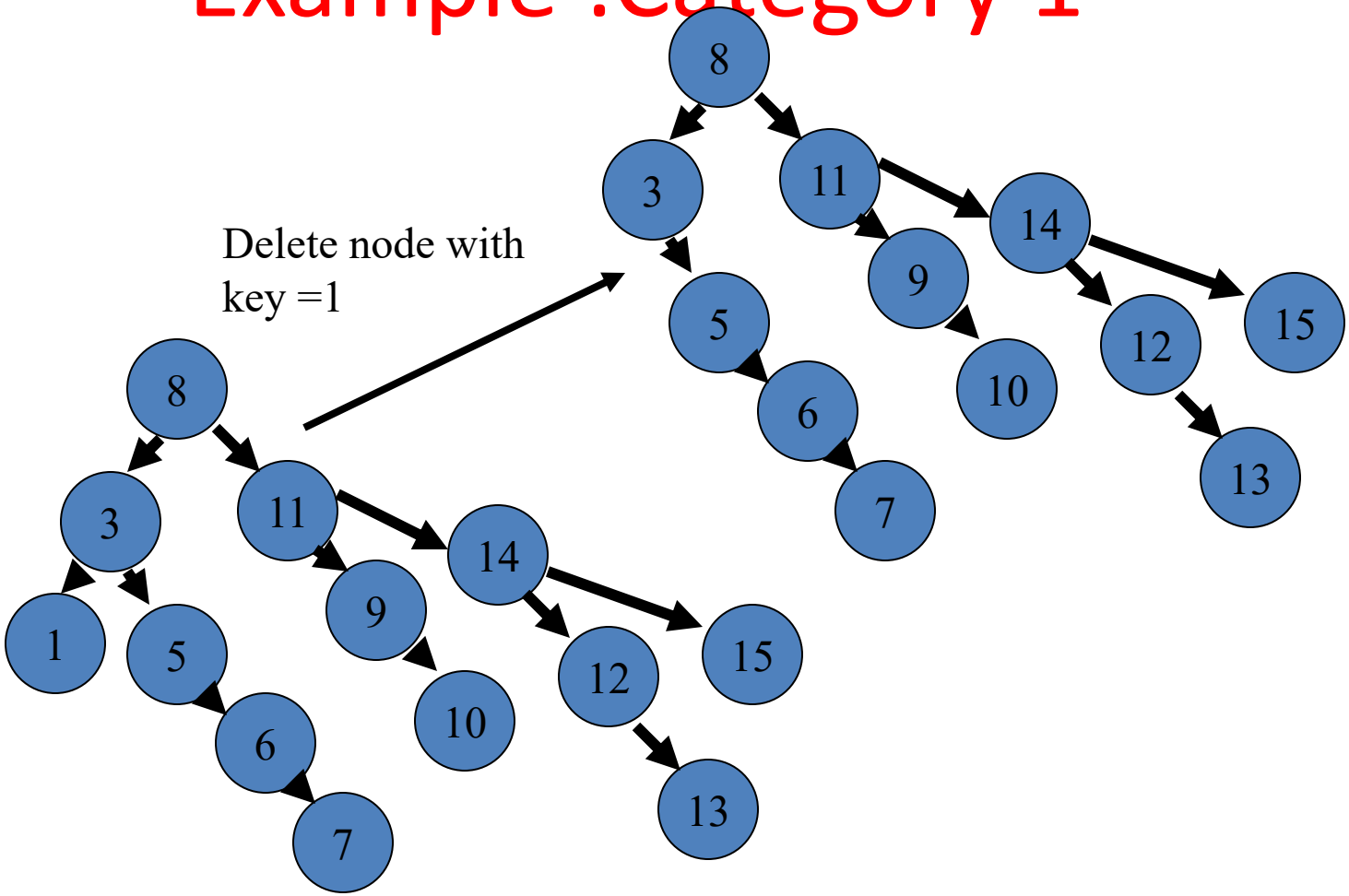
Deleting nodes from a binary search trees.

- There are three categories of nodes we may wish to delete.
- Category 1 : The node to be deleted has no sons.
- - The node can be deleted without any adjustment to the tree.
- - Delete the leaf and set the pointer from its parent (if any) to zero.

Example :Category 1



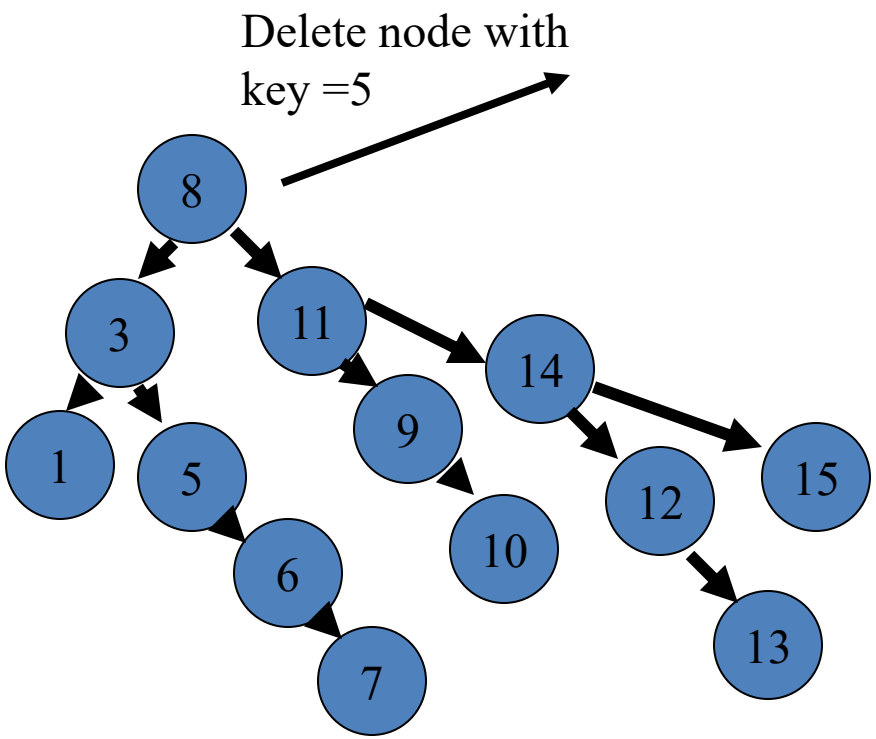
Example :Category 1



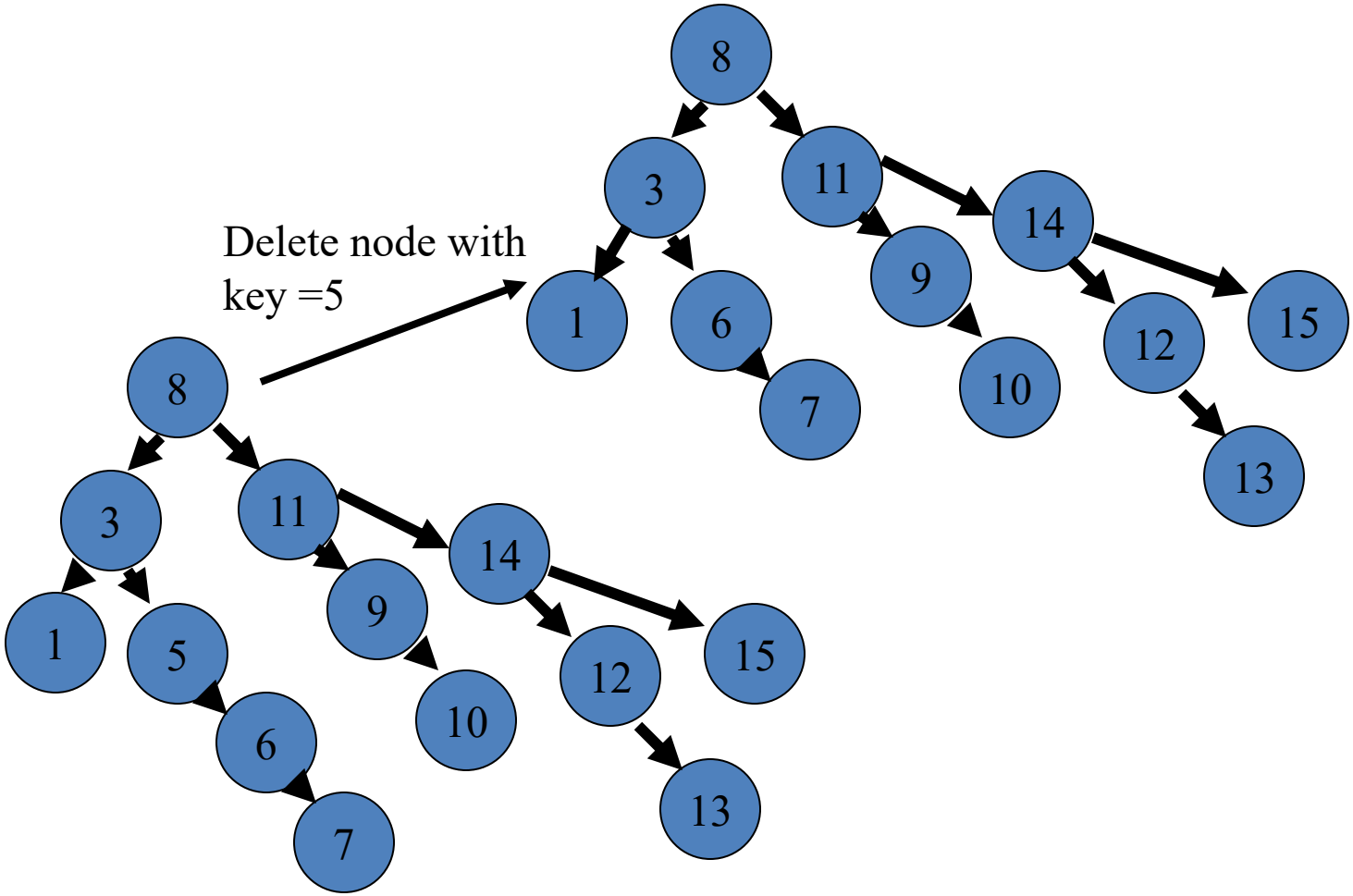
Category 2 : The node to be deleted has only one sub-tree.

- Solution : Its only son can be moved up to its take place.
- We just redirect the references from the node's parent so that it points to the child.

Example :Category 2



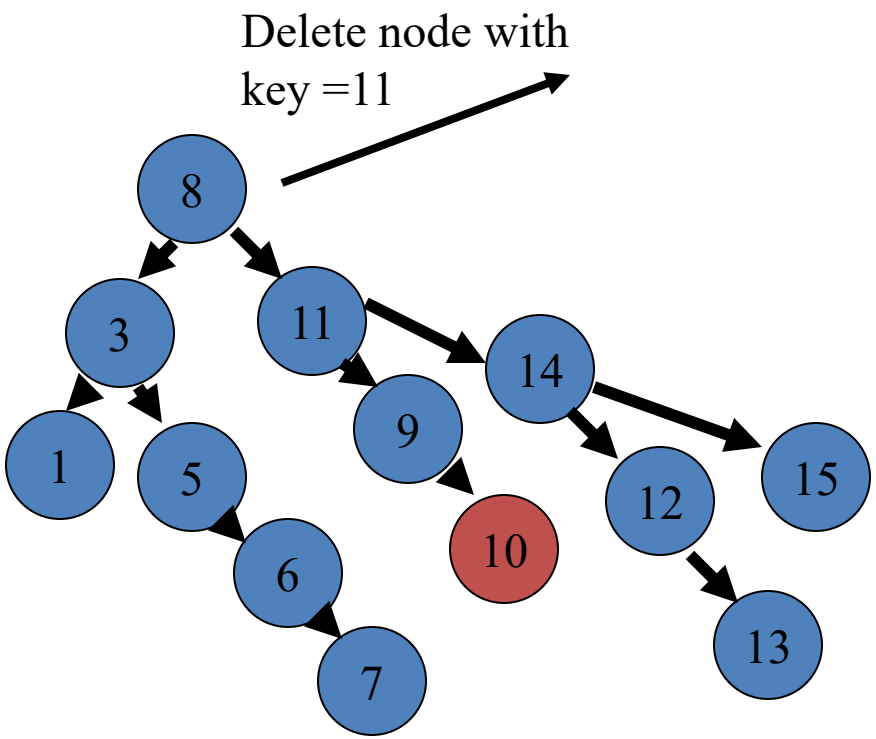
Example :Category 2



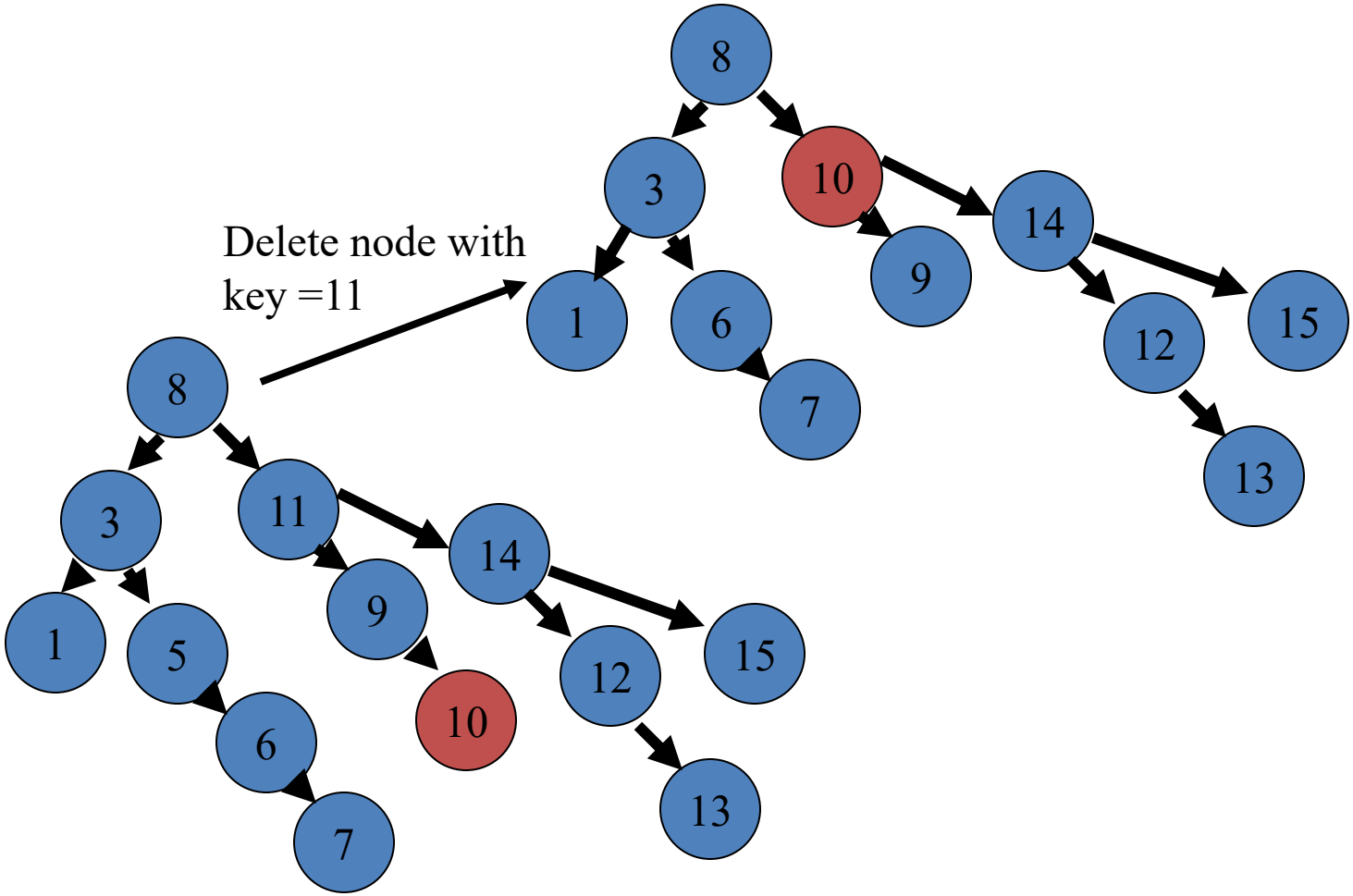
Category 3 : The node to be deleted has two sub-trees.

- Solution : this method we shall use is to replace the node being deleted by the rightmost node in its left sub-tree
- Or Leftmost mode in its right-sub-tree.

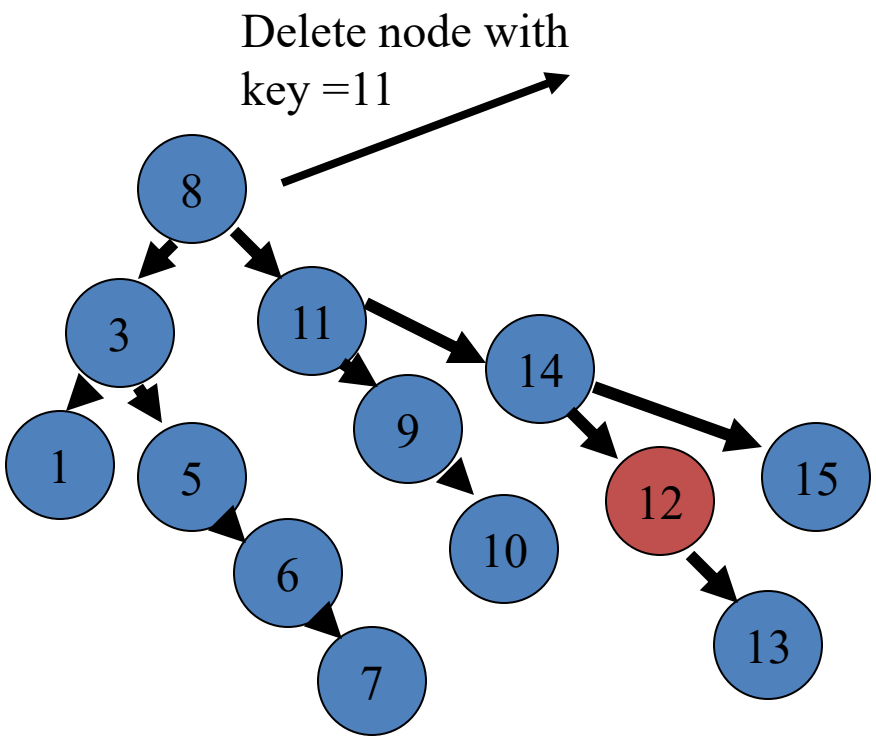
Example :Category 3



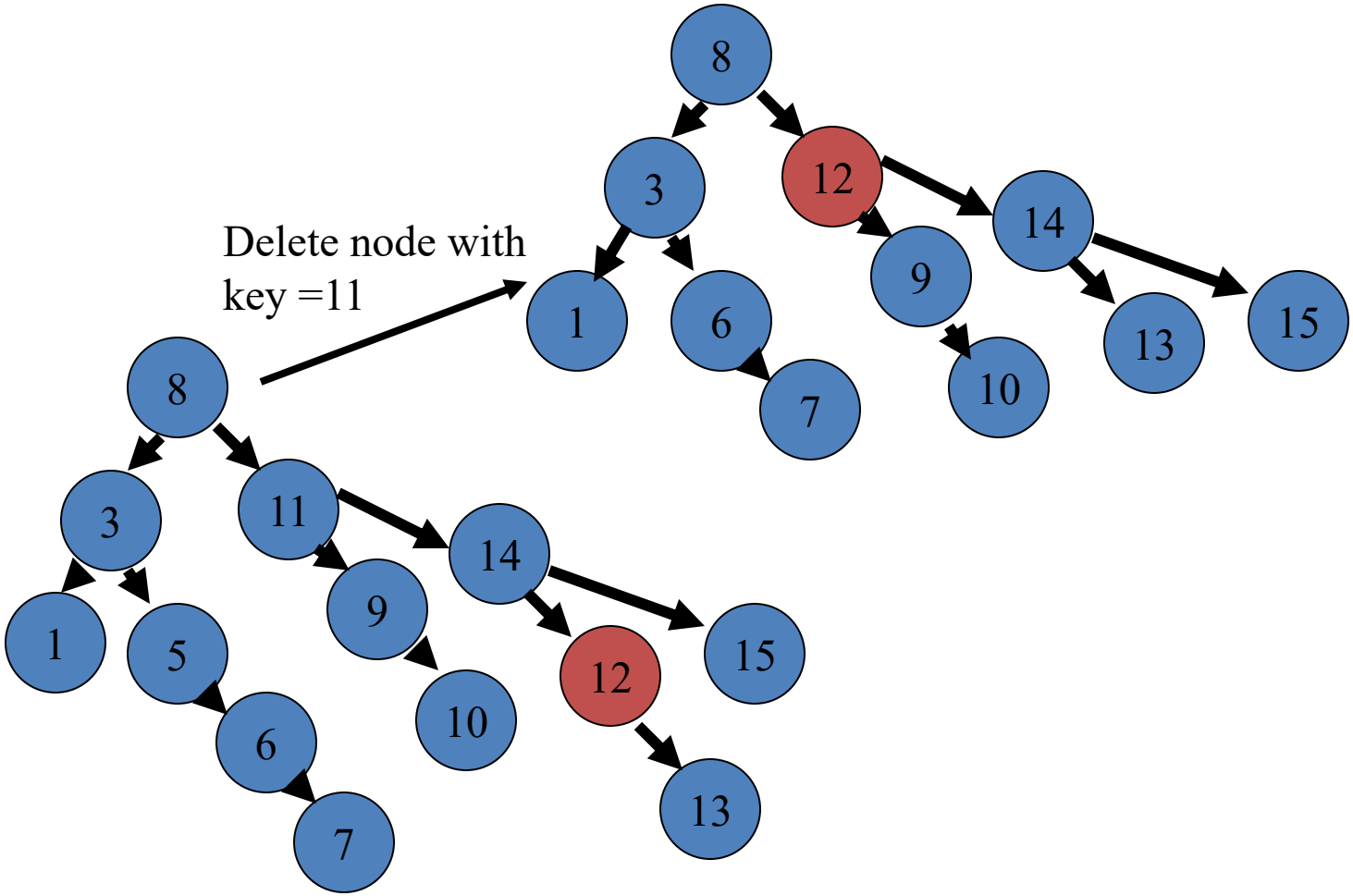
Example :Category 3



Example :Category 3



Example :Category 3



Algorithm to delete a node from a binary search tree

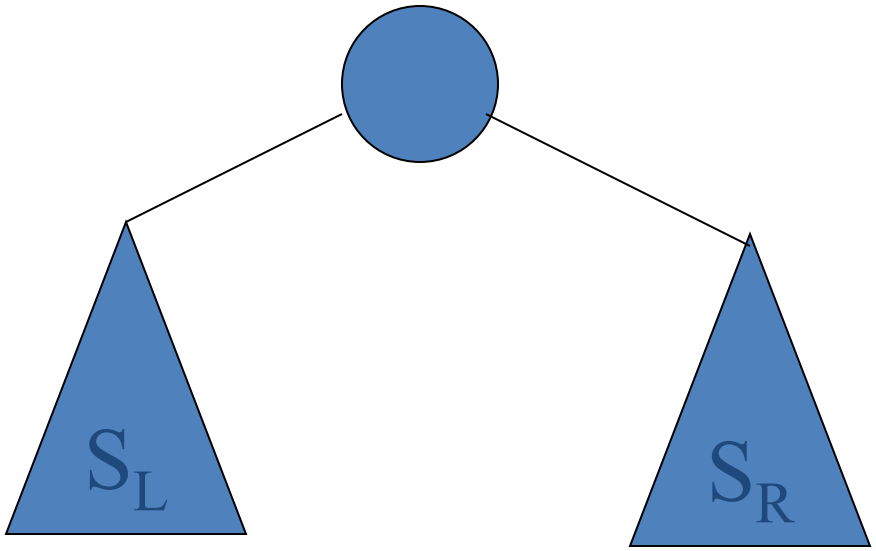
```
Delete (TREE, VAL)

Step 1: IF TREE = NULL
          Write "VAL not found in the tree"
        ELSE IF VAL < TREE -> DATA
          Delete(TREE->LEFT, VAL)
        ELSE IF VAL > TREE -> DATA
          Delete(TREE -> RIGHT, VAL)
        ELSE IF TREE -> LEFT AND TREE -> RIGHT
          SET TEMP = findLargestNode(TREE -> LEFT)
          SET TREE -> DATA = TEMP -> DATA
          Delete(TREE -> LEFT, TEMP -> DATA)
        ELSE
          SET TEMP = TREE
         IF TREE -> LEFT = NULL AND TREE -> RIGHT = NULL
              SET TREE = NULL
          ELSE IF TREE -> LEFT != NULL
               SET TREE = TREE -> LEFT
          ELSE
               SET TREE = TREE -> RIGHT
          [END OF IF]
          FREE TEMP
        [END OF IF]
Step 2: END
```

Determining the Height of a Binary Search Tree

- In order to determine the height of a binary search tree, we calculate the height of the left sub-tree and the right sub-tree.

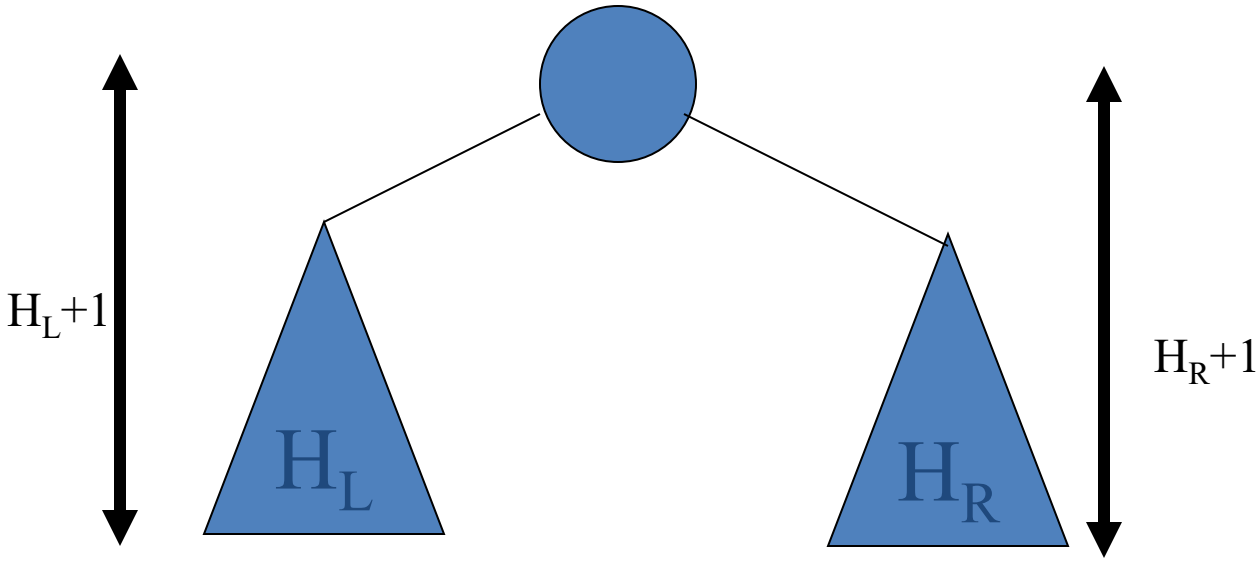
Recursive view used to calculate the size of a tree : $S_T=S_L+S_R+1$



Routine to compute the size of a node

```
//* Return the size of the binary tree rooted at t.
if  (t = = null)
return 0
else
return 1+size(t->left)+ size(t->right)
}
```

Recursive view used to calculate the height of a tree : $H_T = \max(H_L + 1, H_R + 1)$



Routine to compute the height of a node

```
//* Return the height of the binary tree rooted at t.

If (t = = null)
 return 0;
else
return 1+maximum(height(t->left),height(t->right);
}
```

Other operations

- We can perform, find operation by starting at the root and then repeatedly branching either left or right, depending result of a comparison.

Findmin and findmax

- To perform a findmin, we start at root, and repeatedly branch left as long as there is a left child. The stopping point is the smallest element.
- Findmax routine is same, except that branching is to right child.

Findmin operation

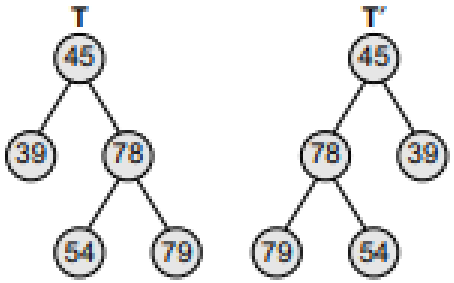
```
If (t = = null)
return null;
else if (t->left= = null)
return t;
return findMin(t->left);
}
```

finMax(Non recursive)

```
{
if (t!=null)
While (t->right!=null)
t=t->right;
return t;
}
```

Finding the Mirror Image of a Binary Search Tree

- Mirror image of a binary search tree is obtained by interchanging the left sub-tree with the right sub-tree at every node of the tree.
- For example, given a tree T, the mirror image of T can be obtained as T_1.
- Consider the following tree.



Finding the Mirror Image of a Binary Search Tree cont….

- A recursive algorithm more convenient to obtain the mirror image of a binary search tree.
- In the algorithm, if TREE != NULL, that is if the current node in the tree has one or more nodes, then the algorithm is recursively called at every node in the tree to swap the nodes in its left and right sub-trees.

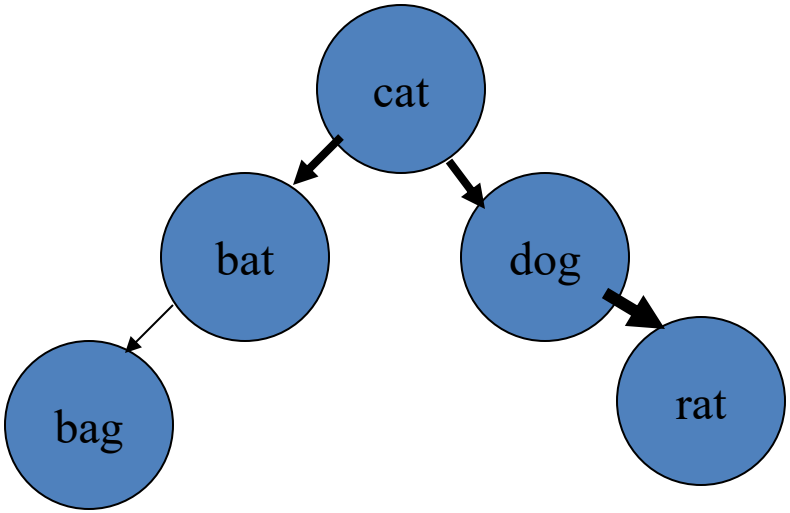
Finding the Mirror Image of a Binary Search Tree cont….

```
MirrorImage(TREE)

Step 1: IF TREE != NULL
            MirrorImage(TREE -> LEFT)
            MirrorImage(TREE -> RIGHT)
            SET TEMP = TREE -> LEFT
            SET TREE -> LEFT = TREE -> RIGHT
            SET TREE -> RIGHT = TEMP
        [END OF IF]
Step 2: END
```

Application : Implementation of a Dictionary [words]

- Eg : cat, bat, dog,rat,bag etc..



Homework

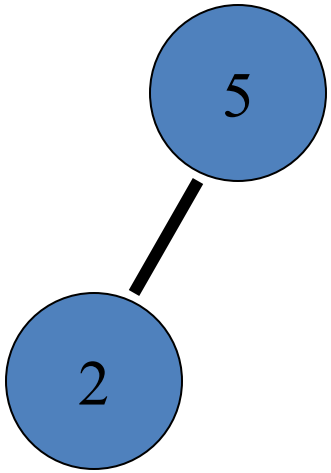
- Write a program to create a binary search tree and perform all the operations discussed in the preceding sections.

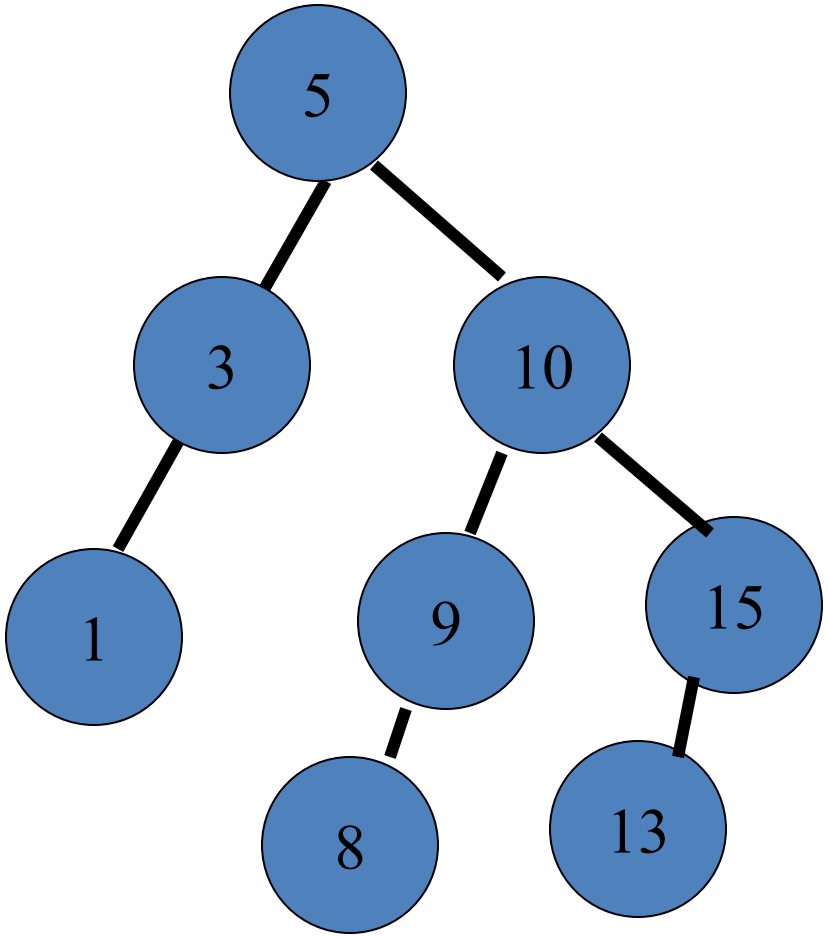
AVL Trees

AVL trees

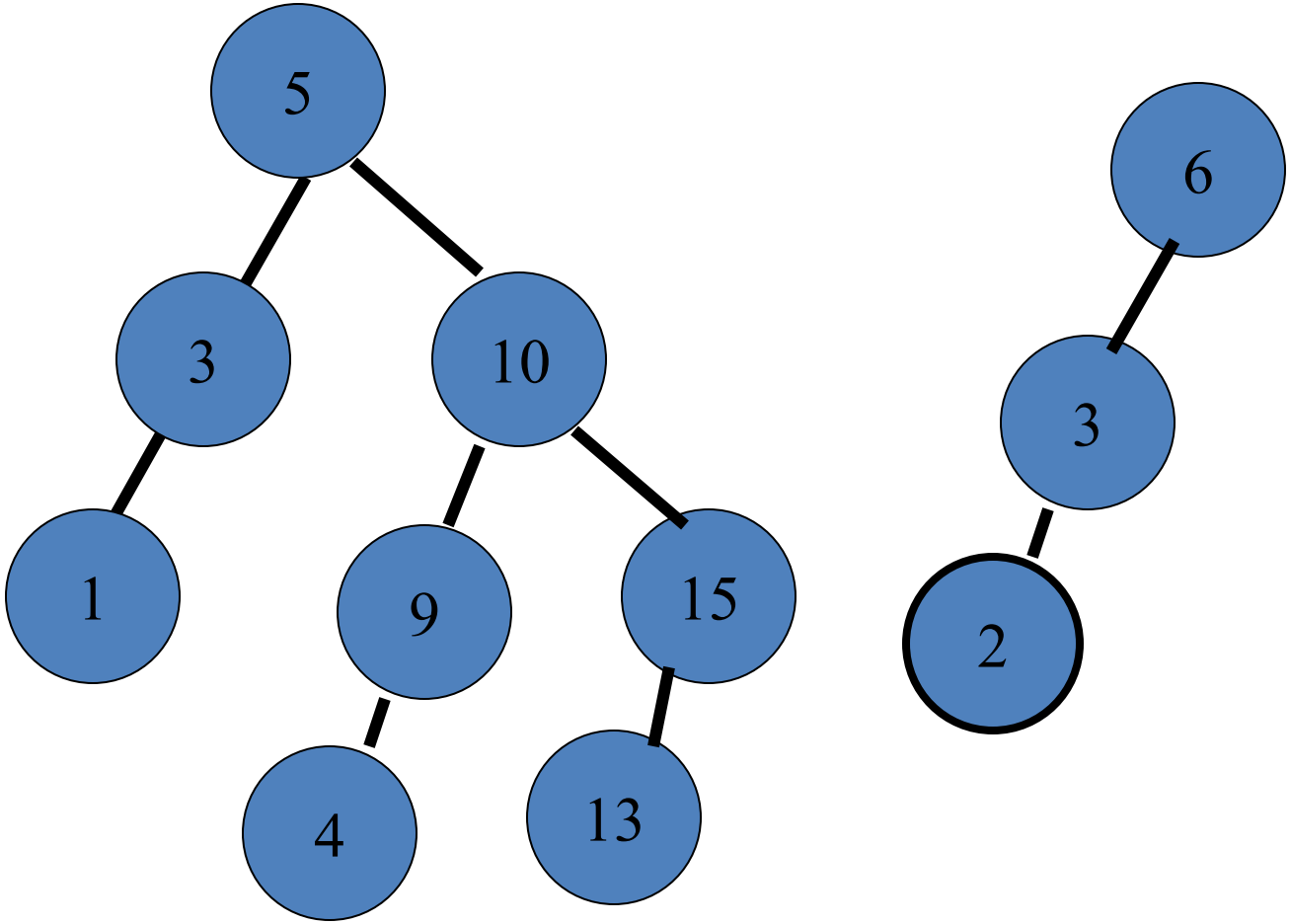
- AVL tree is a self-balancing binary search tree invented by G.M. A delson-Velsky and E.M. Landis in 1962
- Definition :
- An AVL tree is a binary search tree with the additional balance property that, for any node in the tree, the height of the left and right sub-trees can be defer by at most 1.
- *Balance factor = Height (left sub-tree) – Height (right sub-tree)*
- The height of the empty sub-tree is –1
- Balance factor = |Height of the left sub-tree –height of the right sub-tree |
- **-1=< balance factor <=1**

AVL Trees ?





AVL trees ?



Algorithm : Inserting new nodes into an AVL trees

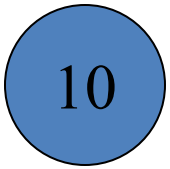
1. Insert the node in the same way as in an ordinary binary search tree.
2. Beginning with the new node, trace a path back towards the root, checking the difference in height of the sub-trees at each node along the way.
3. If you find a node with an imbalance (a height difference other than 0,+1,-1), stop your trace at this point.
4. Consider the node with the imbalance and two nodes on the layers immediately below this point on the path back to the new node.

Algorithm : Inserting new nodes into an AVL trees

5. If these three nodes lie in a straight line, apply a single rotation to correct the imbalance.
6. If these three nodes lie in a dog-leg pattern, apply a double rotation to correct the imbalance.

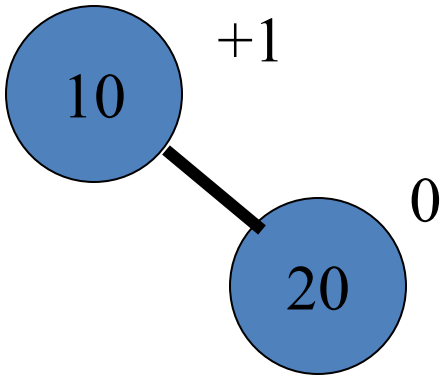
Construction of an AVL trees.

- Eg. 10,20,30,25,27,7,4,23,26,21
- We begin by inserting integer 10 into the root

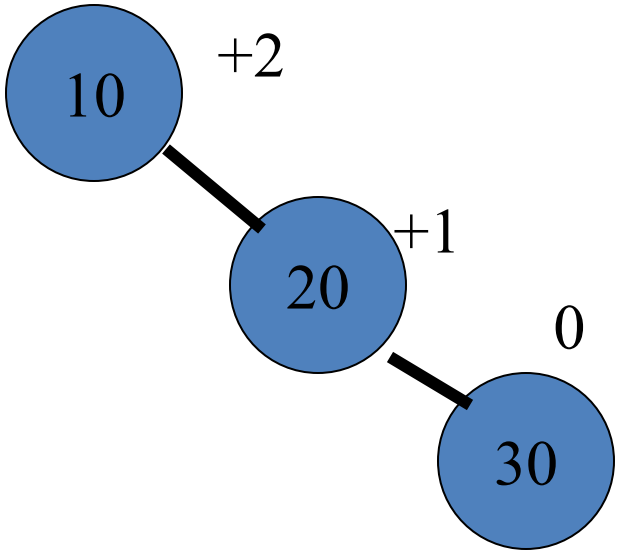


- This node satisfies the AVL condition.

Now we add another node 20



Now we add third node, 30



The root node 10 has a deference of 2, which violates the AVL conditions.

Therefore, we must re-arrange the node to restore the balance in the tree.

We perform an operation known as the rotation when the tree goes out of balance. There are two types of rotation used in AVL trees. **Single rotation and Double rotation.**

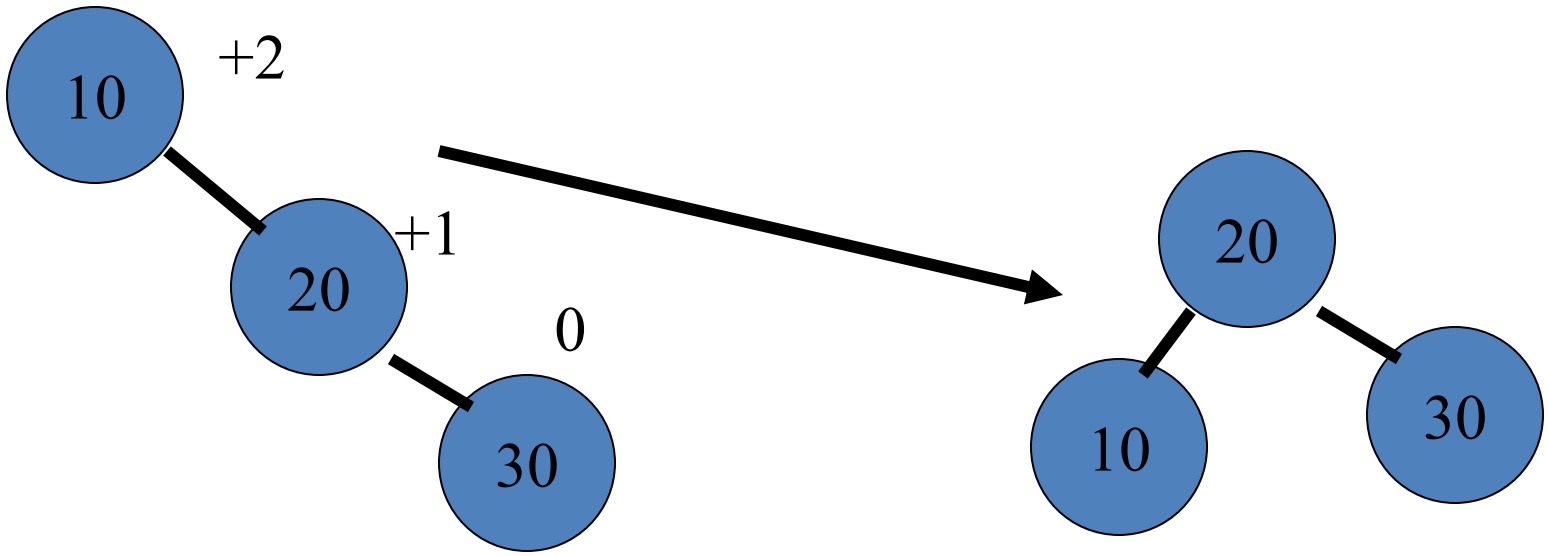
The rules for deciding which type of rotations to be used.

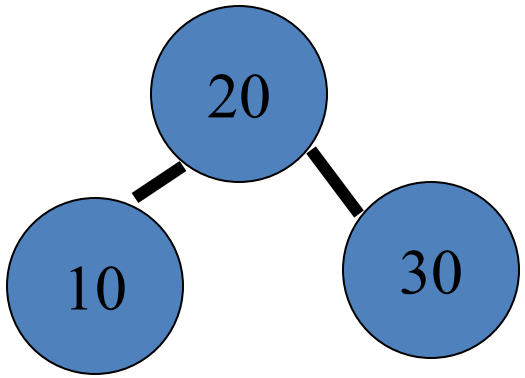
1. When you have found the first node is out of balance, restrict your attention to that node and the two nodes in the two layers immediately below it.
2. If these three nodes lie in a straight line, single rotation is needed to restore the balance.
3. If these three nodes lie in a “dog-leg” pattern You need double rotation to restore the balance.

In our example. Node 10 is imbalance , consider two layers immediately below this node. These nodes lie in a straight line. So we need a single rotation to restore the balance

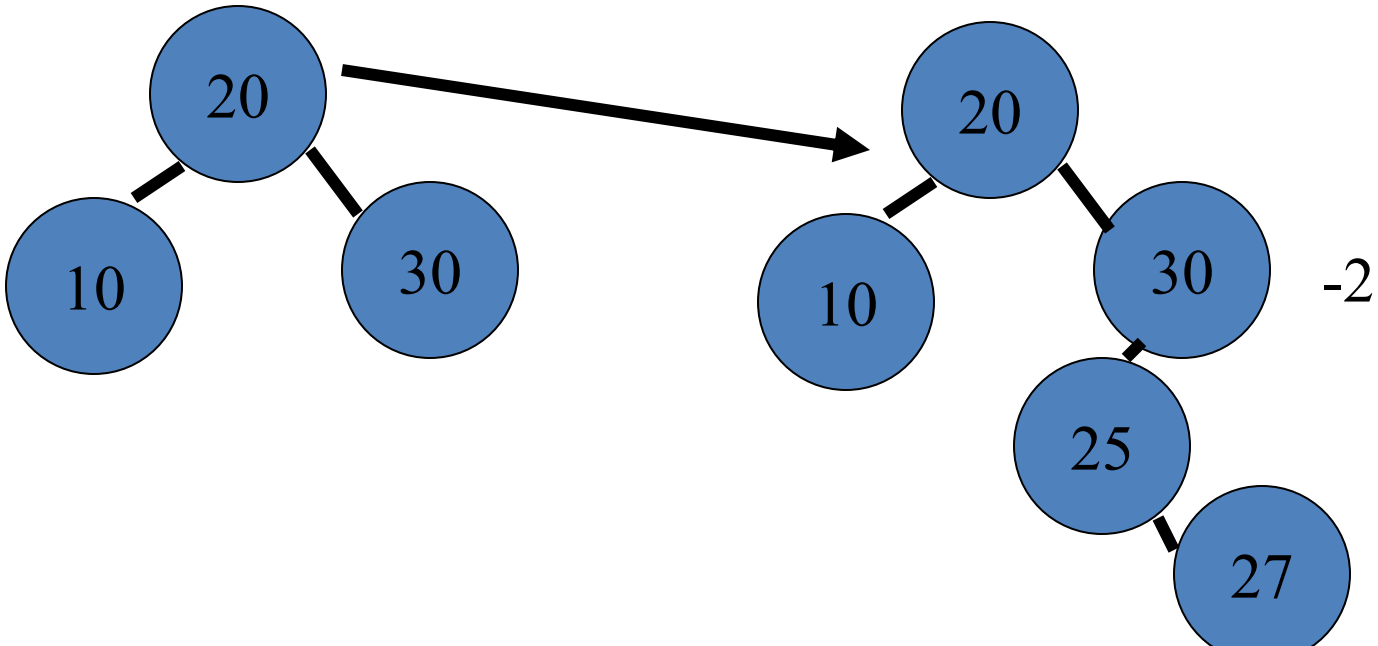
Single Rotation :

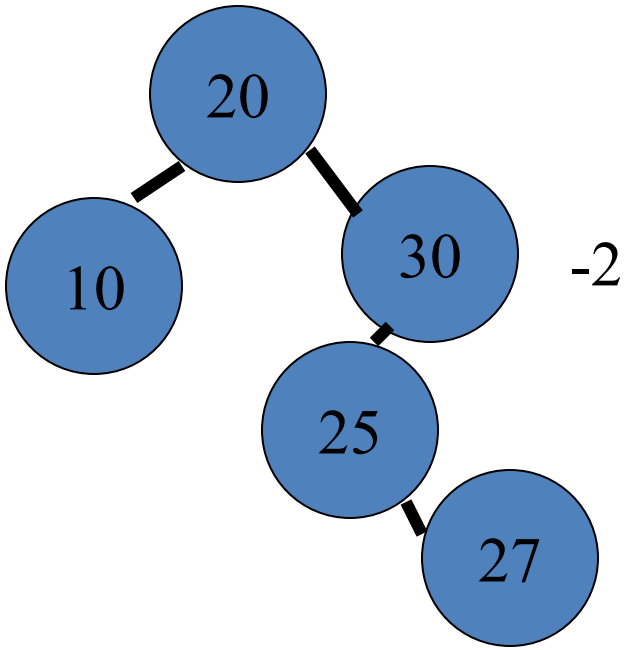
- This involves shifting the middle node up to replace the top node down to become the left child of the middle.





We continue by adding two more nodes 25 & 27



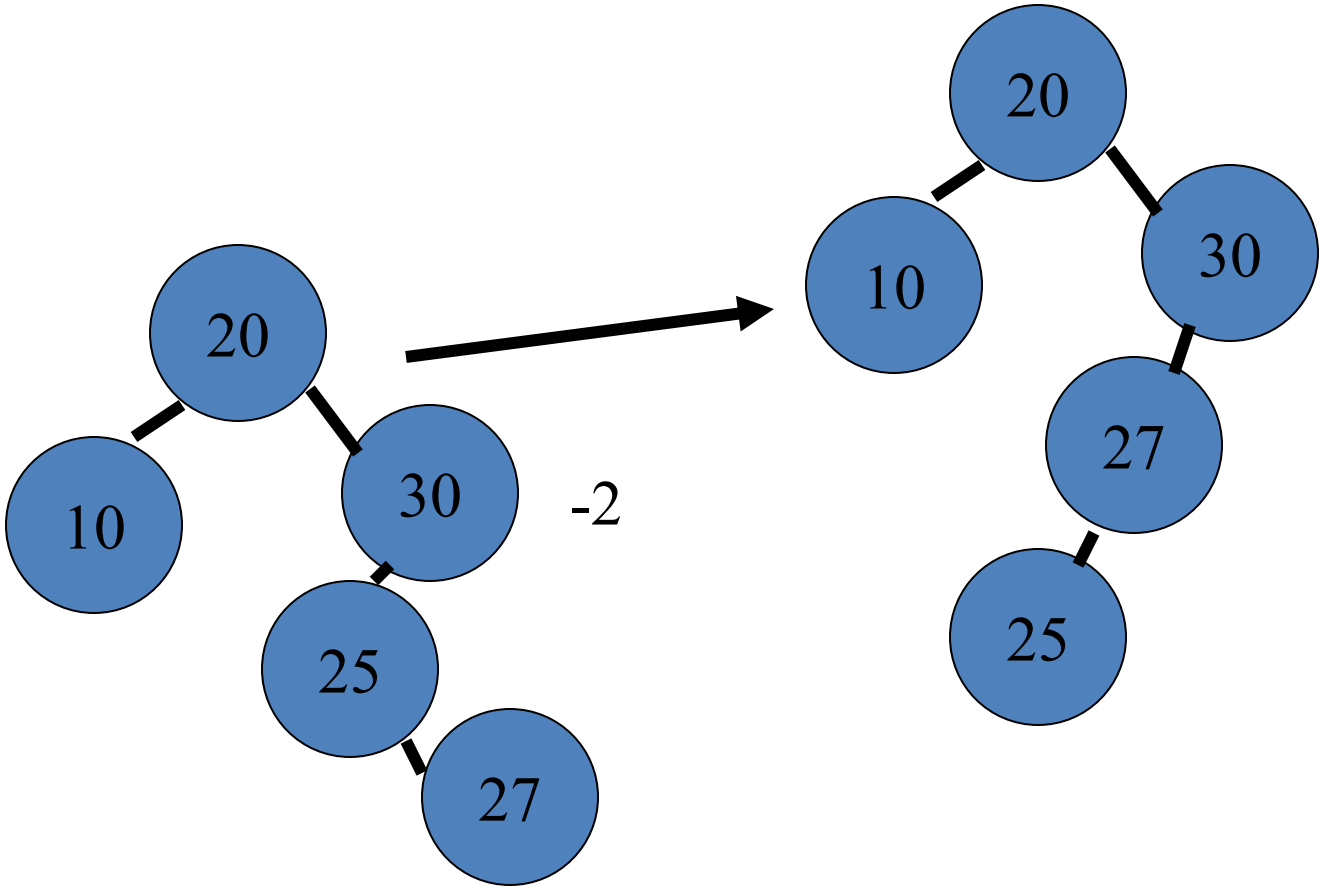


The first imbalance is detected at node 30 these three nodes form a dog-leg pattern.

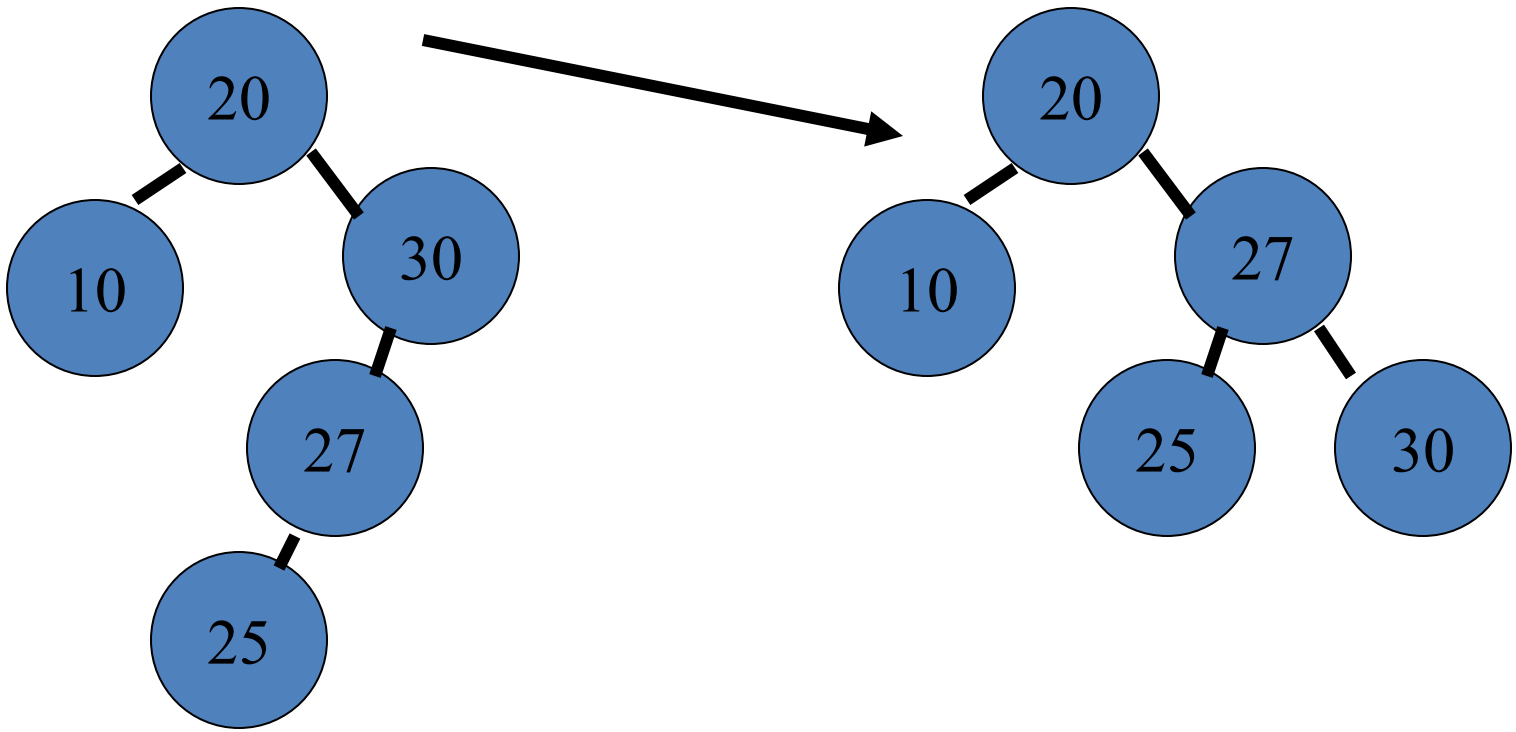
We require a double rotation to correct the balance.

Double rotation consists of two single rotations. These two rotations are in opposite directions.

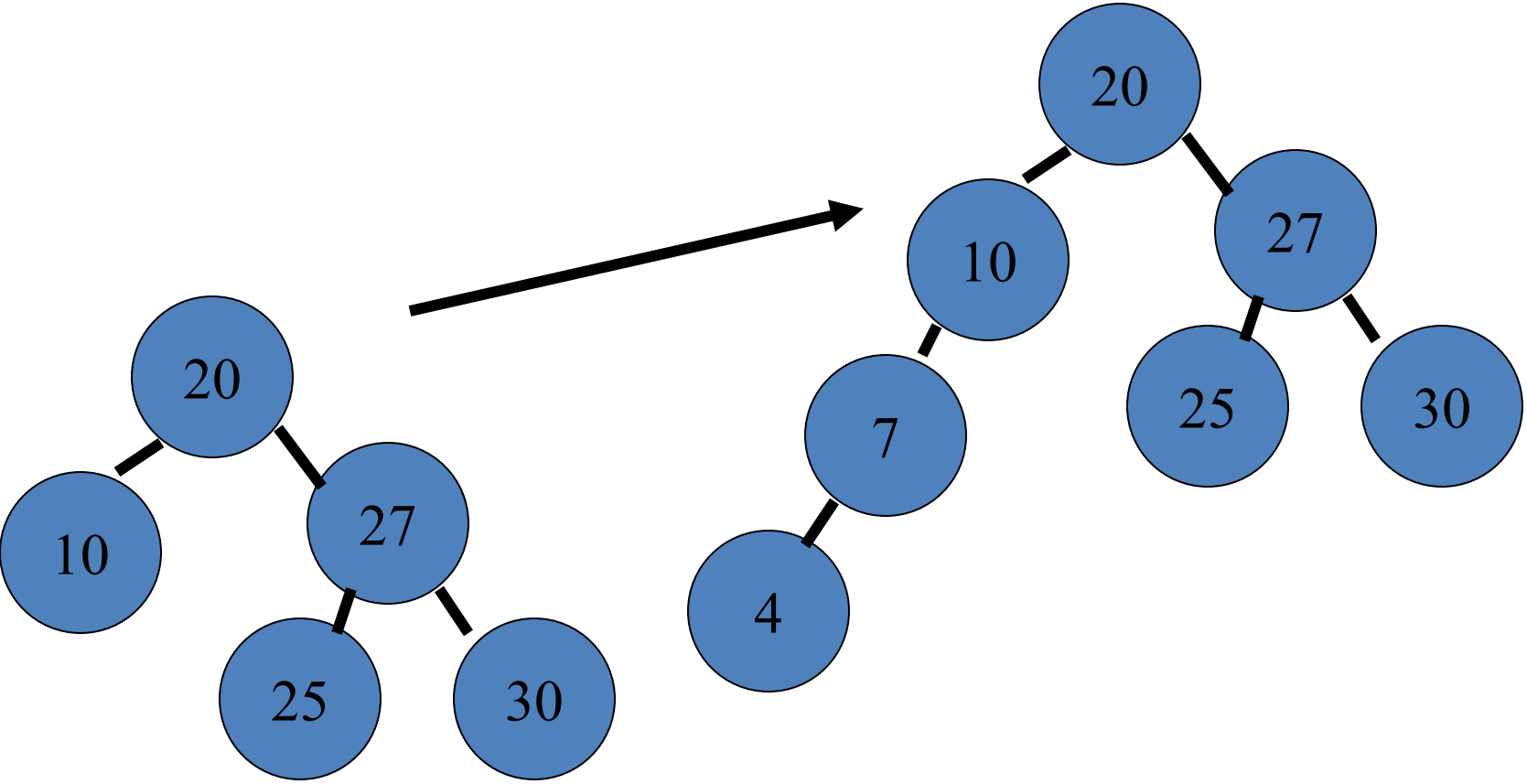
<u>1St Rotation</u>: we rotate the node 27 up to 25, and 25 down to become the left child of 27.



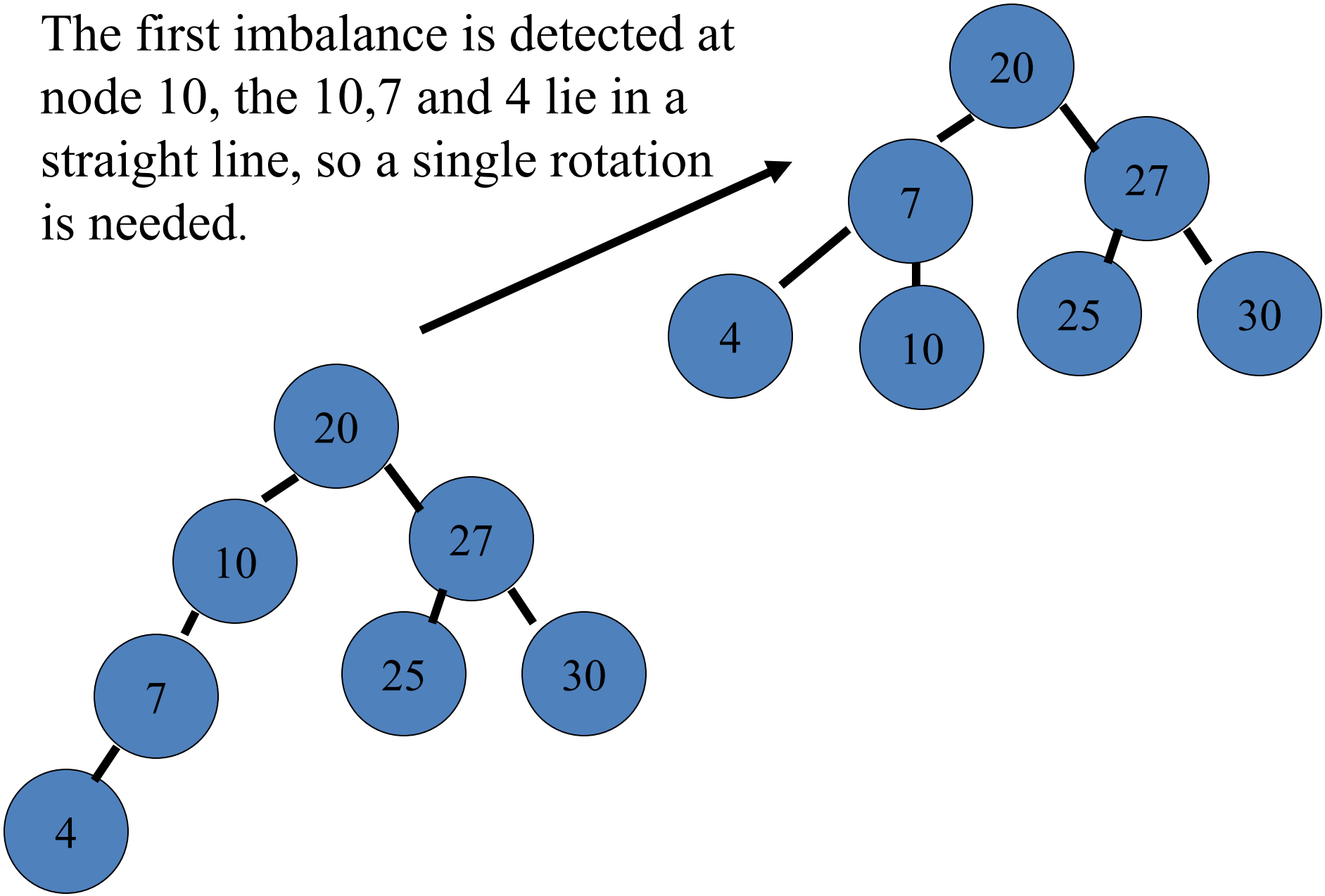
2nd Rotation : Node 27 rotates up to replace 30 and node 30 rotates down to become the left child of 27



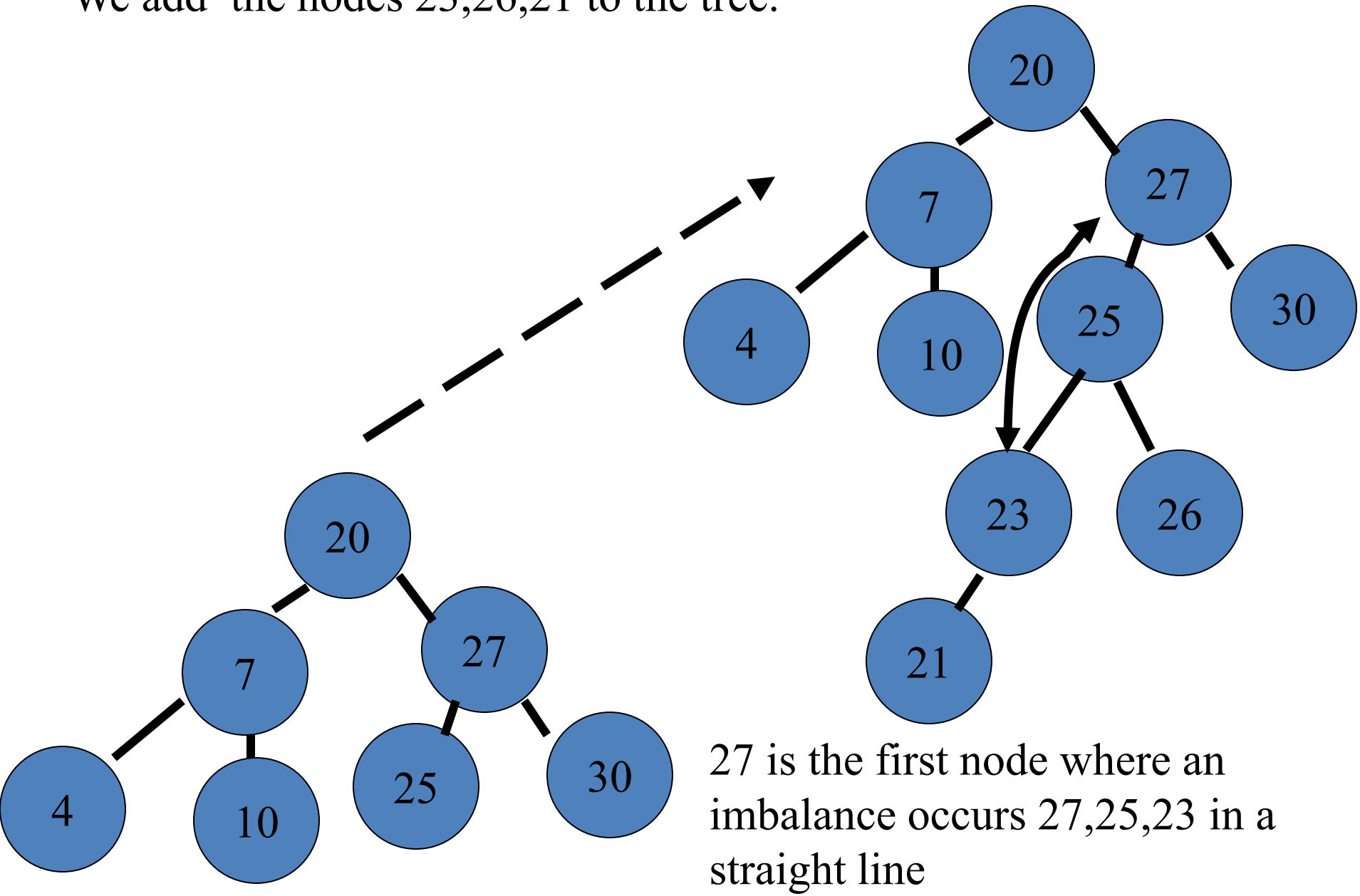
We continue by adding the nodes 7 and 4 to the tree.



The first imbalance is detected at node 10, the 10,7 and 4 lie in a straight line, so a single rotation is needed.

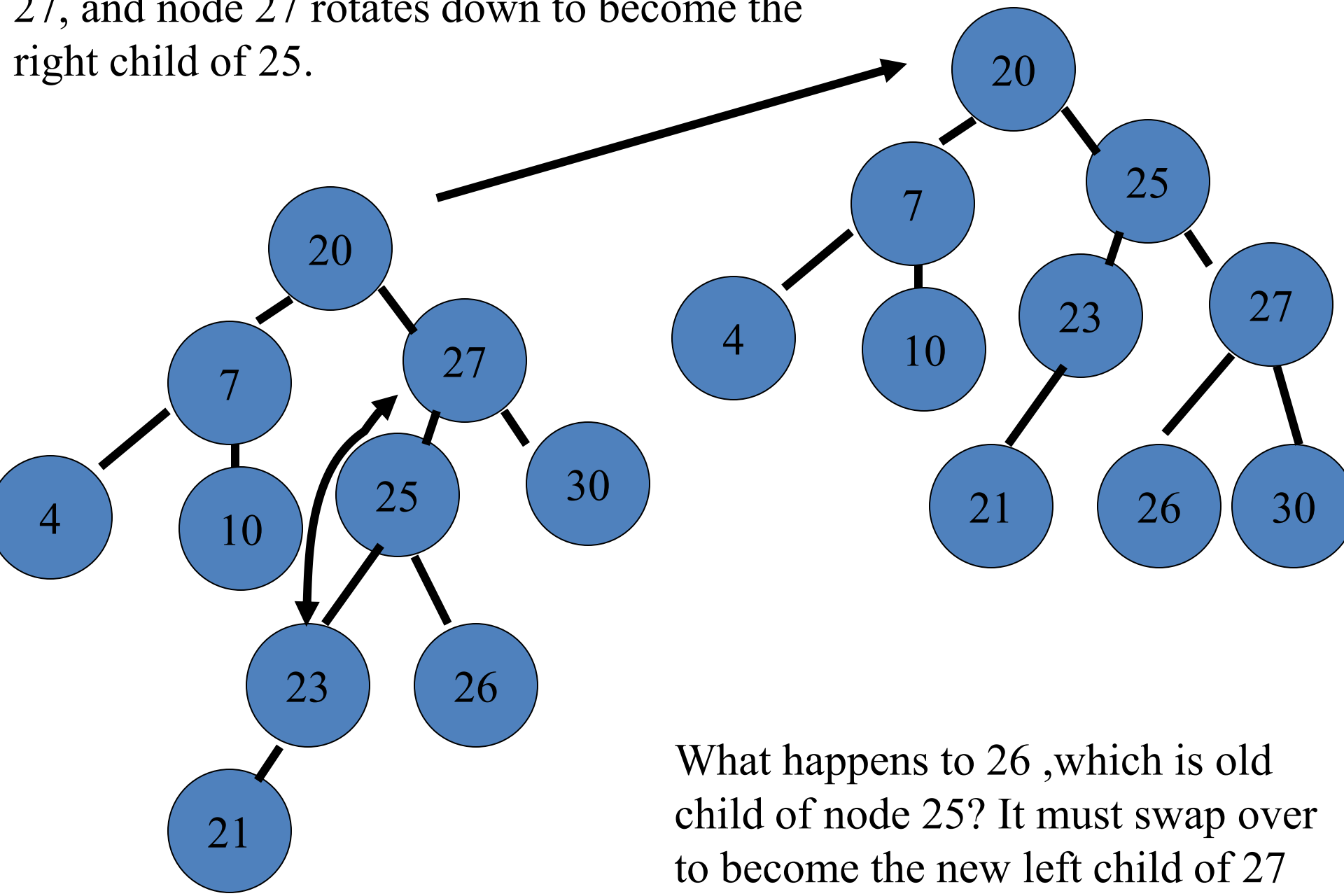


We add the nodes 23,26,21 to the tree.



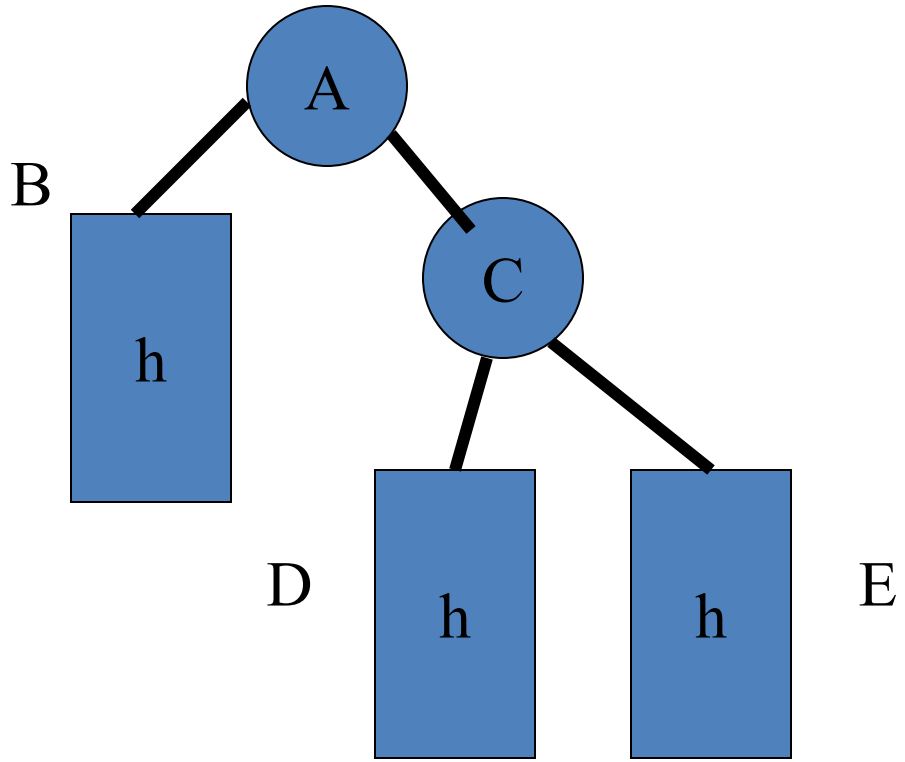
27 is the first node where an imbalance occurs 27,25,23 in a straight line

The middle node (25) rotate up to replace node 27, and node 27 rotates down to become the right child of 25.



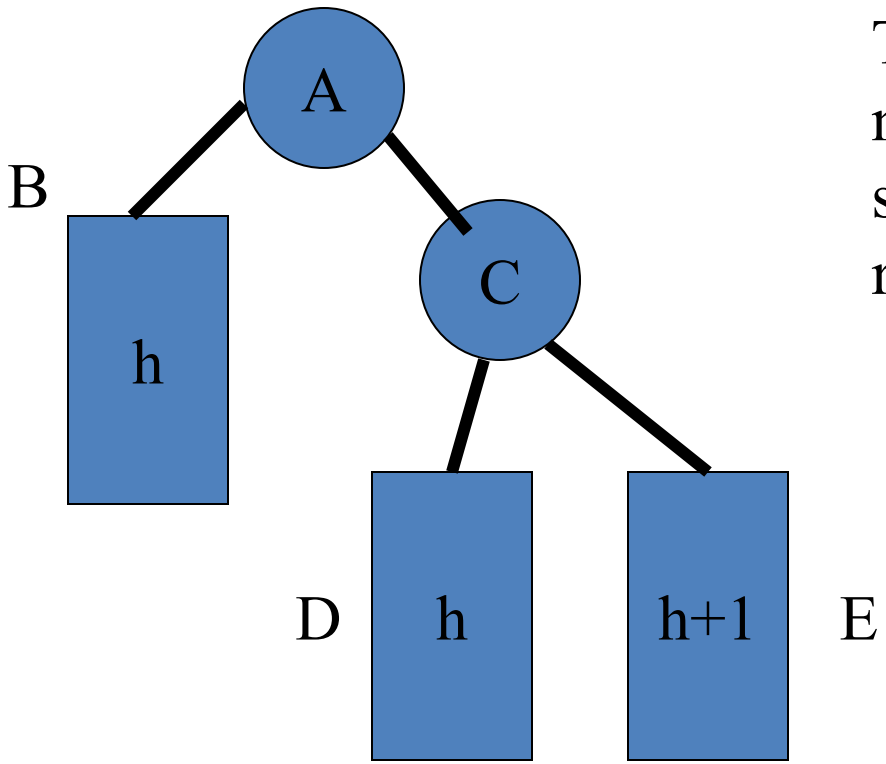
What happens to 26 ,which is old child of node 25? It must swap over to become the new left child of 27

Single Rotation



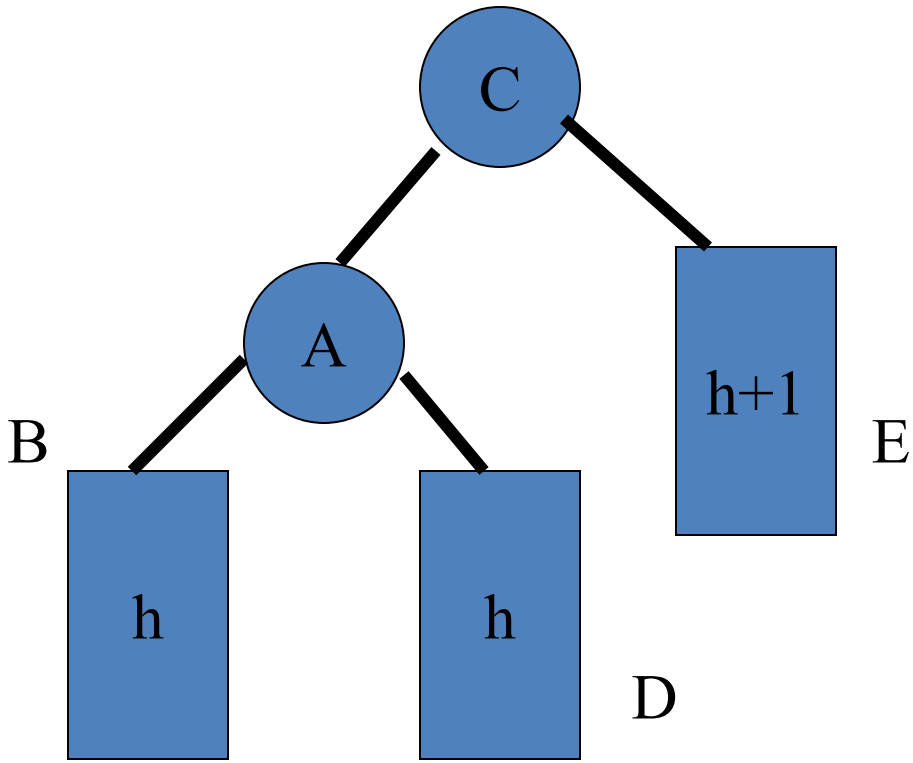
Suppose we want to insert a node into the sub-tree under node E

The tree is imbalance at node A



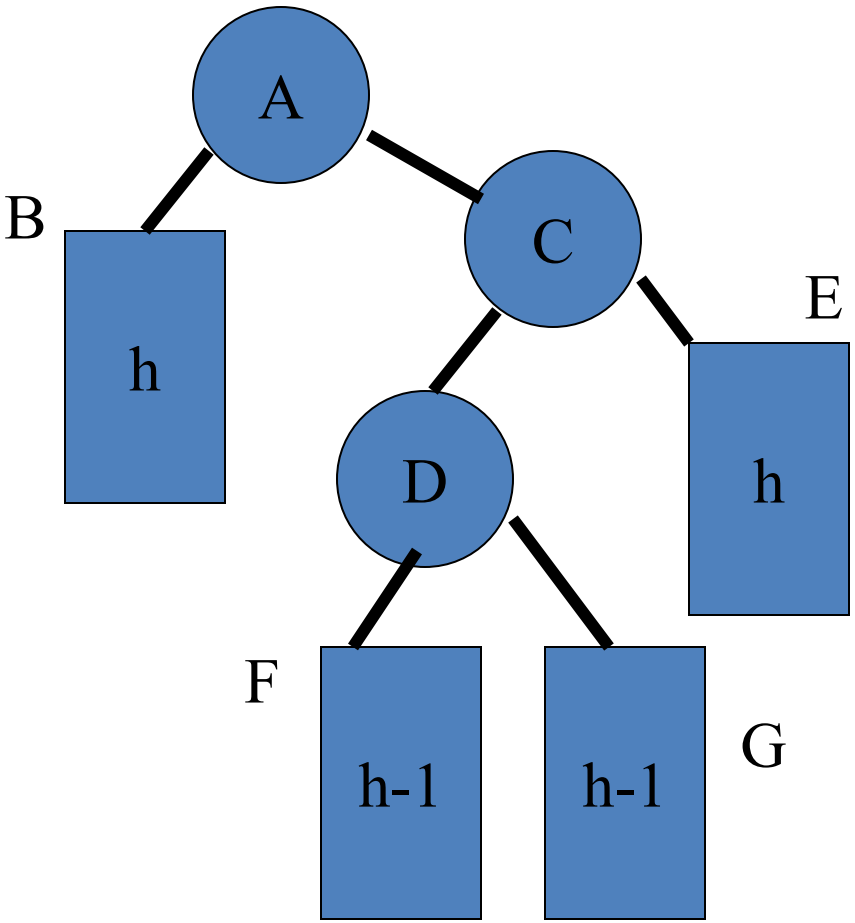
The two nodes immediately below node A. Nodes A,C,E are in the straight line. So we need a single rotation.

We rotate C up to replace A and A down become the left child of C. Node D must swap over to become the new child of A



This tree is balanced.

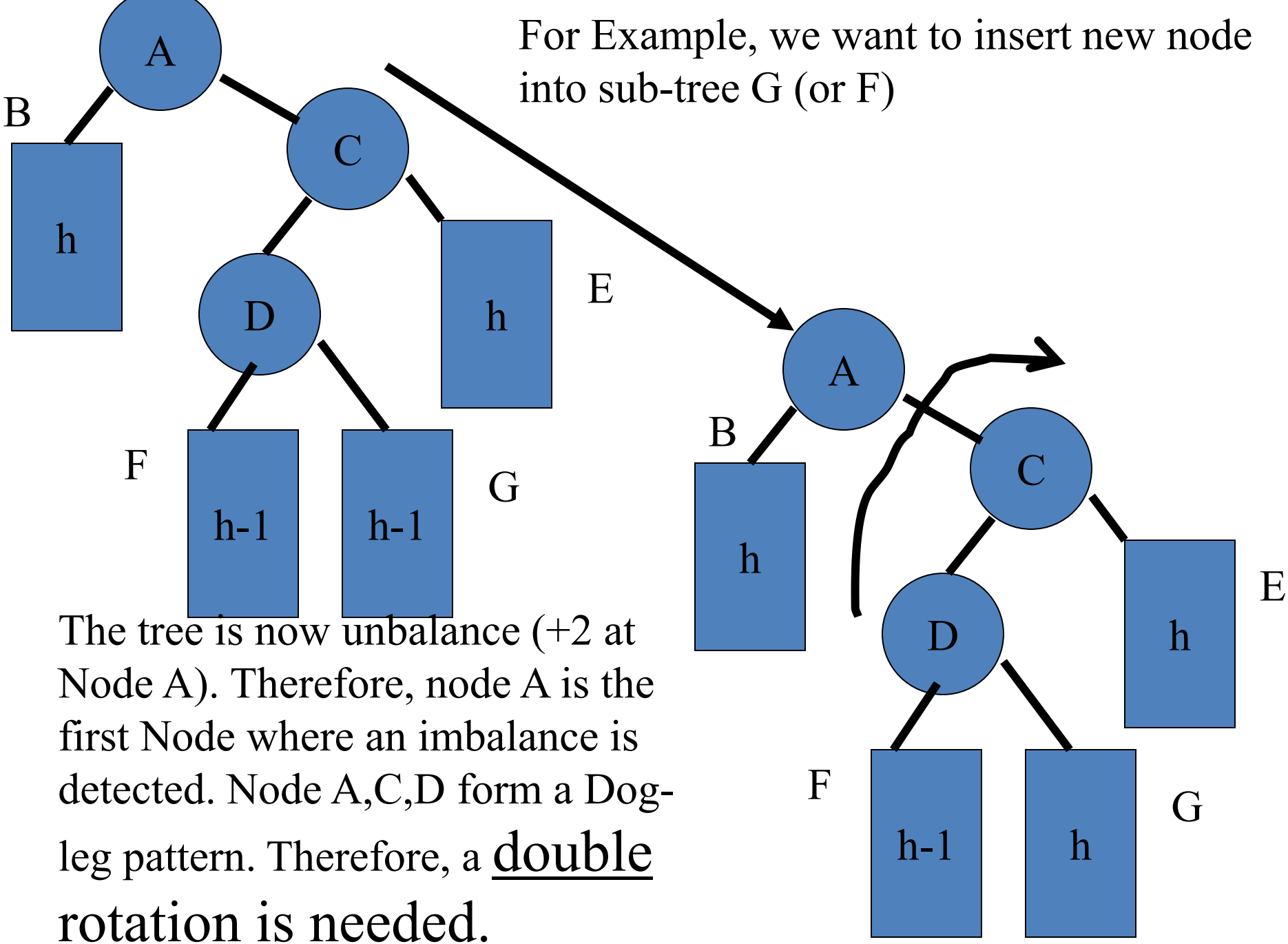
Double Rotation



This tree is an AVL tree, Assuming all sub-trees in the rectangular boxes are AVL trees. The height differences are 0 at node D & C and +1 at Node A.

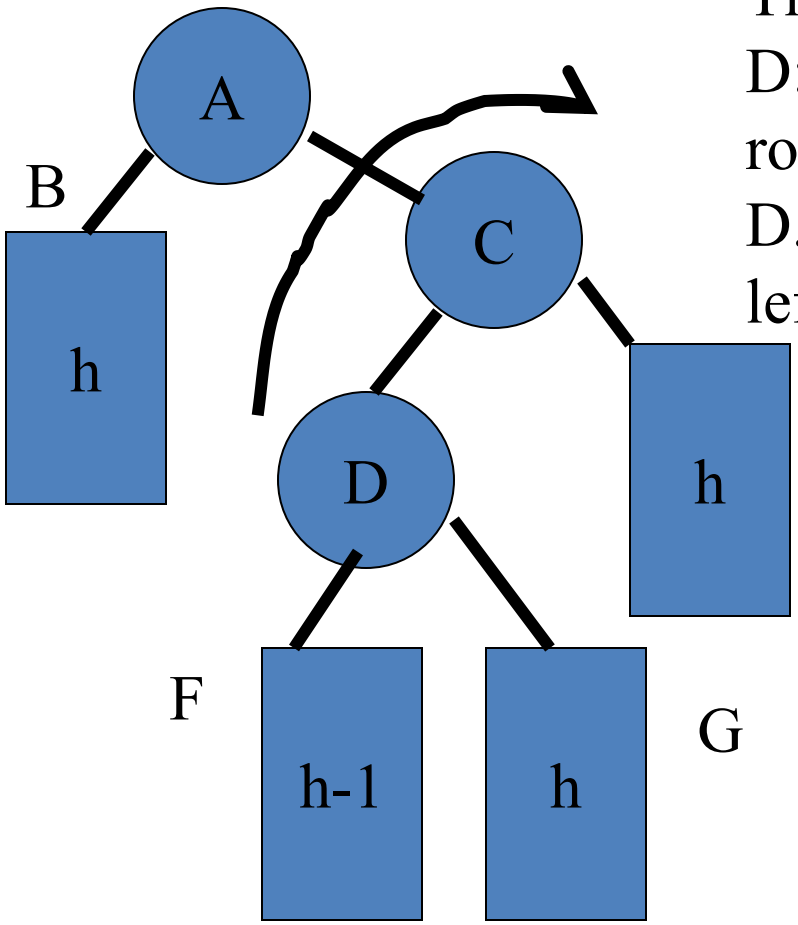
For Example, we want to insert new node into sub-tree G (or F)

For Example, we want to insert new node into sub-tree G (or F)

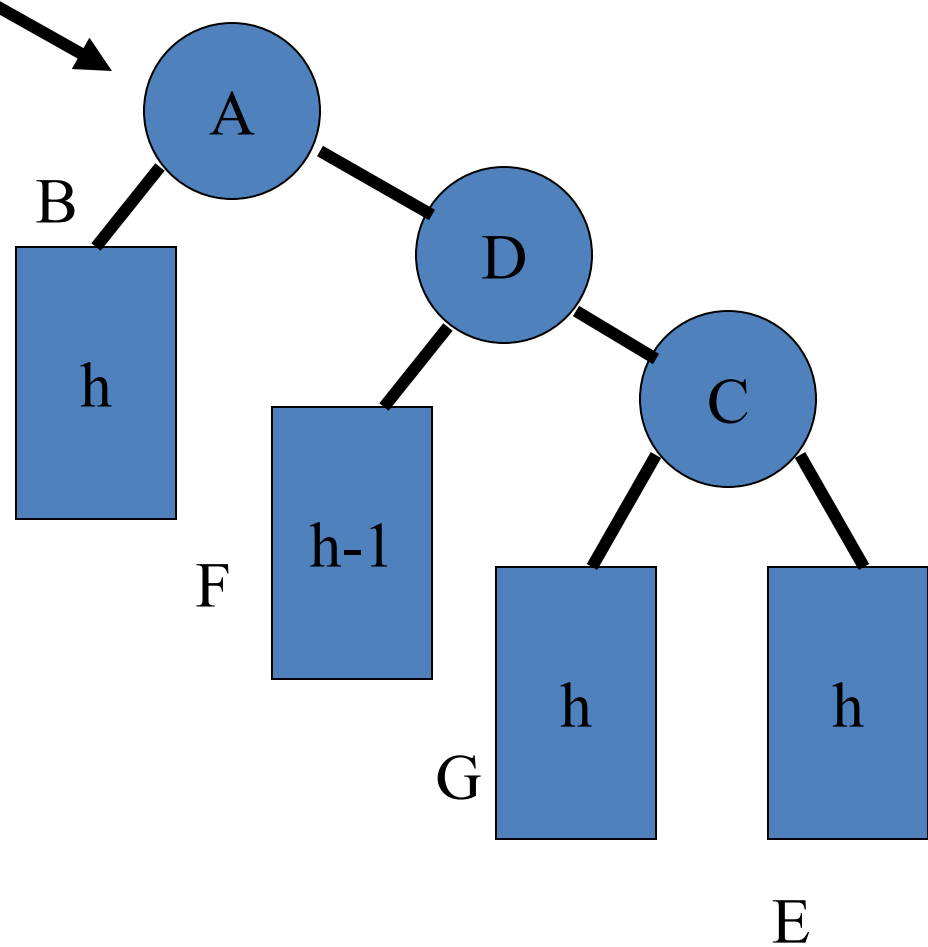


The tree is now unbalance (+2 at Node A). Therefore, node A is the first Node where an imbalance is detected. Node A,C,D form a Dog-leg pattern. Therefore, a <u>double rotation is needed.</u>

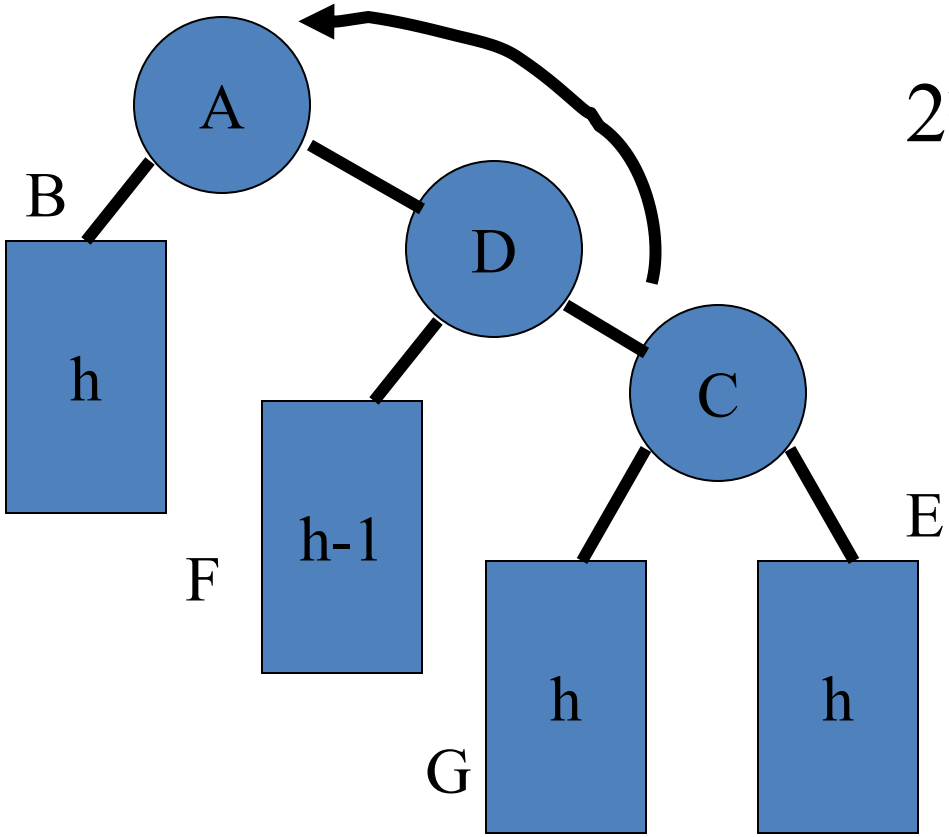
The first rotation involves node C and Node D: Node D rotates up to replace C and C rotates down to become new right child of D. Sub-tree G swaps over to become the left child of node C.



1st rotation

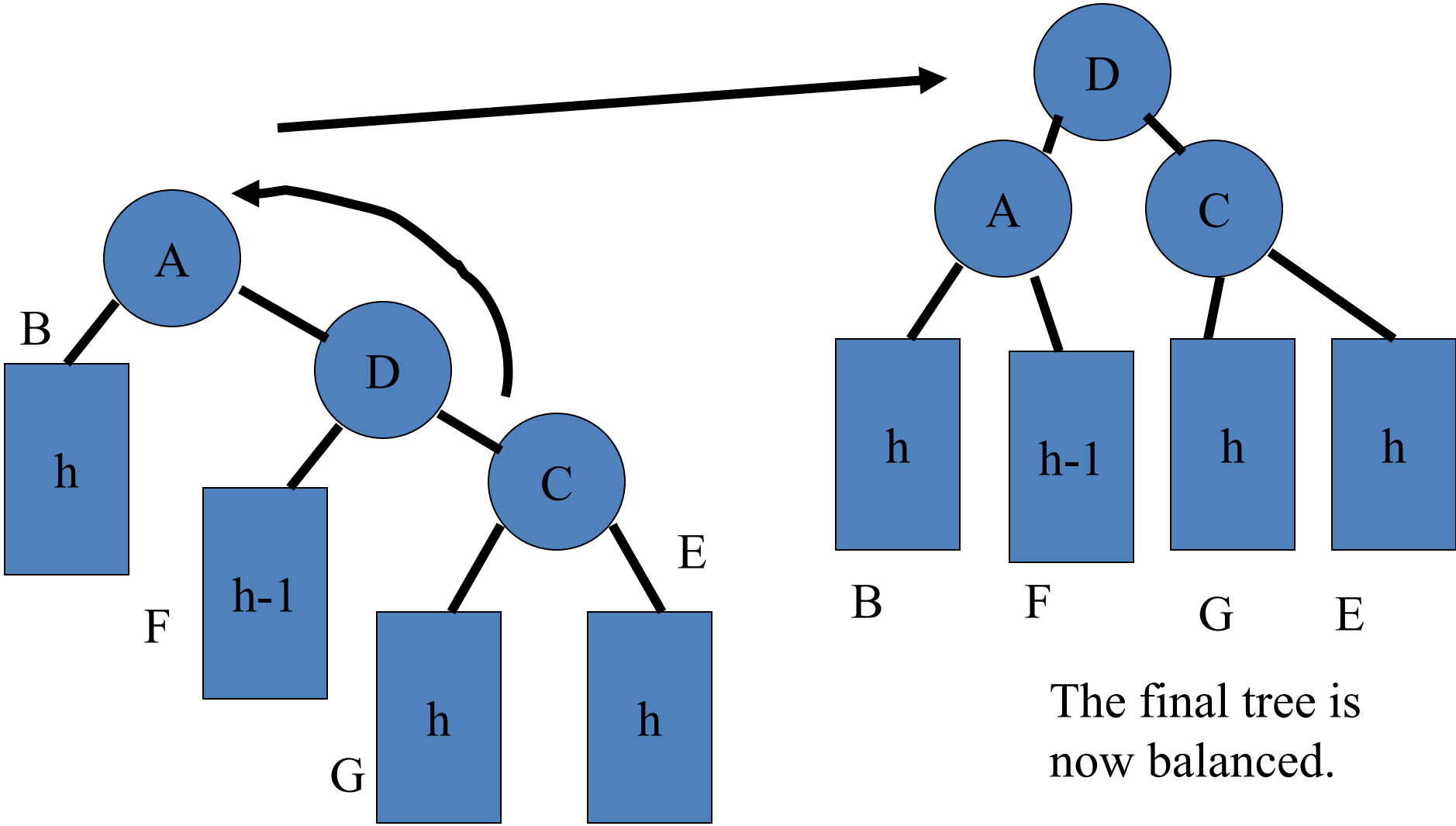


This first rotation does not solve the imbalance problem since the height differences are still +2 at both nodes A and D.



2nd Rotation

Therefore, we perform the 2nd rotation, which is in the opposite direction to the first one, and which involves nodes A,D and C



The final tree is now balanced.

Node D rotates up to replace A. Node A rotates down to become the left child of D, and Sub-tree F swaps over to become the new right child of node A.

<u>AVL- Tree implementation</u>

Node declaration for AVL trees

- Home work:
- Write a program that shows insertion operation in an AVL tree

- Data structures using Java, **Mark** Allen **Weiss**